Neural Networks

Lesson 9 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

27 November 2026 · reading time about 100 minutes

Contents

Lesson plan . . . 2
1. Why this lesson exists . . . 3
    1.1 Meridian Instruments, and one problem that is not theirs . . . 3
2. One neuron, one line . . . 4
    2.1 The perceptron, and the unit this course already met . . . 4
    2.2 What one line is worth on the acceptance data . . . 5
3. A hidden layer: fences made of lines . . . 7
    3.1 The smallest problem that needs one . . . 7
    3.2 A network built by hand . . . 7
    3.3 What the hidden layer actually did . . . 8
    3.4 Capacity is not the same as findability . . . 9
    3.5 Universal approximation, and what it does not promise . . . 9
4. Forward propagation . . . 10
    4.1 Notation, and why the shapes matter . . . 10
5. Backpropagation . . . 11
    5.1 The idea, before any algebra . . . 11
    5.2 The output layer, and why sigmoid and cross-entropy belong together . . . 11
    5.3 Weights, biases, and the step to the layer below . . . 12
    5.4 The algorithm, and what it costs . . . 13
    5.5 Check the gradient before trusting it . . . 13
6. More than two classes . . . 14
    6.1 Softmax . . . 14
    6.2 The same cancellation, one dimension up . . . 15
    6.3 Does a hidden layer help on digits? . . . 15
7. Activations, and the vanishing gradient . . . 16
    7.1 Why a non-linearity is needed at all . . . 16
    7.2 The sigmoid's derivative is bounded by one quarter . . . 17
    7.3 What that costs, measured . . . 18
    7.4 The predictable mistake: the ReLU has its own failure . . . 20
8. Initialisation . . . 20
    8.1 Zero cannot work, and the reason is not what it first appears . . . 20
    8.2 How large? Propagating the variance . . . 22
9. Optimisation in practice . . . 23

9.1 The learning rate . . . 23
9.2 Mini-batches . . . 25
9.3 Momentum and Adam . . . 25
10. Regularisation, and knowing whether it worked . . . 26
10.1 A network that memorises, and generalises anyway . . . 26
10.2 Three interventions . . . 27
10.3 Whether any of it is distinguishable . . . 28
10.4 The intervention that is not a hyperparameter . . . 29
11. How many units? Capacity against optimisation . . . 29
11.1 A yardstick made of lines . . . 29
11.2 What the sweep shows . . . 30
11.3 The two ceilings, and the identity connecting them . . . 33
12. Choosing, and what carries into lesson 10 . . . 33
Summary . . . 34
Homework . . . 34
Notation used in this lesson . . . 34
Further reading . . . 35

Lesson plan

Time	Minutes	Segment	Material
0:00–0:10	10	Exercise 8 discussed; one neuron is one line	Slides 2–8
0:10–0:30	20	The acceptance problem, and why a line cannot solve it	Slides 9–16
0:30–0:45	15	A hidden layer: fences made of lines; a network built by hand	Slides 17–27
0:45–1:00	15	Forward propagation, shapes, and the loss	Slides 28–33
1:00–1:12	12	**Break**	Slide 34
1:12–1:30	18	Backpropagation derived; checking the gradient	Slides 35–43
1:30–1:52	22	**Notebook 01** — backpropagation from scratch	Slide 44
1:52–2:07	15	Softmax, activations, and the vanishing gradient	Slides 45–54
2:07–2:27	20	**Notebook 02** — Keras, softmax, depth	Slide 55
2:27–2:40	13	Initialisation, optimisers, regularisation	Slides 56–67

Time	Minutes	Segment	Material
2:40–2:58	18	**Notebook 03** — training in practice	Slide 68
2:58–3:00	2	Homework	Slides 69–70
	180	**Total**	**70 slides, 3 notebooks**

---

1. Why this lesson exists

Every model in this course so far has drawn a boundary of a shape fixed in advance. Logistic regression draws a straight line. A decision tree draws boxes with axis-parallel sides. A support vector machine with a radial kernel draws something curved, but the curve's family was chosen by whoever picked the kernel.

A neural network draws a boundary whose *shape is itself learned*. That is the one idea in this lesson, and everything else — backpropagation, activation functions, initialisation, optimisers, dropout — exists either to make that learning possible or to stop it going wrong.

It is worth saying at the outset what this lesson is not. It is not a survey of architectures, and it contains no result that requires a graphics card. Three hours is enough to derive backpropagation properly, implement it, and meet the handful of failure modes that account for most of the time anyone spends debugging a network. That is a better use of the time than a tour.

1.1 Meridian Instruments, and one problem that is not theirs

Two of the three datasets come from **Meridian Instruments**, a fictional maker of optical distance sensors, and the third is real.

1. **Acceptance testing.** Every sensor off the line is measured on two calibration axes — a gain offset in decibels and a phase offset in degrees. The unit passes when both offsets are *jointly* small enough for the firmware's single global correction to absorb them. The accept region is therefore the inside of a closed curve, and no straight line encloses anything. Notebooks 01 and 03.
2. **Two-channel drift.** A sensor's two channels drift with temperature. When they drift the same way the drift is common mode and one correction removes it; when they drift oppositely it is differential and nothing does. So a unit is correctable exactly when the two drifts share a sign — the **exclusive-or (XOR)** function, the smallest problem that needs a hidden layer. Notebook 01.
3. **Handwritten digits.** The 8×8 digit images bundled with scikit-learn: 1,797 of them, ten classes, real data. Notebooks 02 and 03.

The third is there for a reason that matters more than variety. On the acceptance data a linear model scores 0.55 and a network 0.94; on the digits a linear model scores 0.93 and the best network here 0.97. **A hidden layer is not always the answer**, and a lesson that only showed problems where networks win would teach the opposite.

The file that generates the two synthetic datasets, `instrument_data.py` in the `Notebooks` folder, publishes their rules as `TRUE_*` constants. In particular the acceptance test rig records the wrong

verdict for 3% of units, which puts a **ceiling of 0.97** on every score in this lesson — a number to read every accuracy against, rather than reading it against 1.

> **Try this:** before reading on, open `instrument_data.py` and read `make_acceptance_test`. Given that both tolerances are the same multiple of their axis's production spread (the standard deviation across units), what shape is the accept region after each axis is standardised? Section 2 has the answer, and the figure.

---

2. One neuron, one line

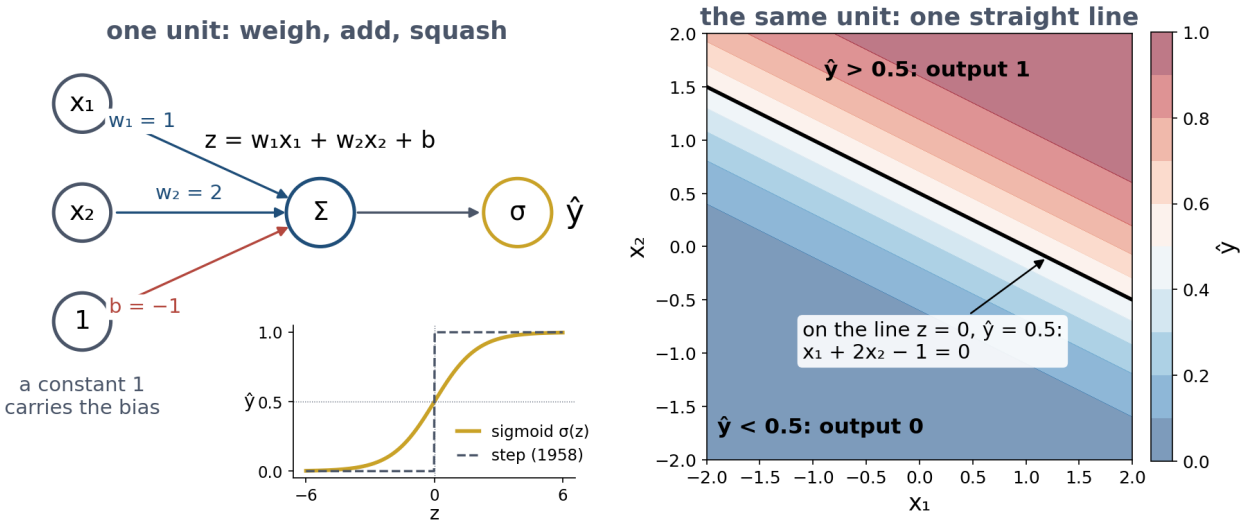
2.1 The perceptron, and the unit this course already met

The **perceptron** (Rosenblatt, 1958) is the ancestor: weights w, a bias b, and the rule "output 1 if $w^\top x + b > 0$, otherwise 0". Replace the hard threshold with a sigmoid and you have exactly the logistic regression unit of lesson 4:

$$\hat{y} = \sigma(w^\top x + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

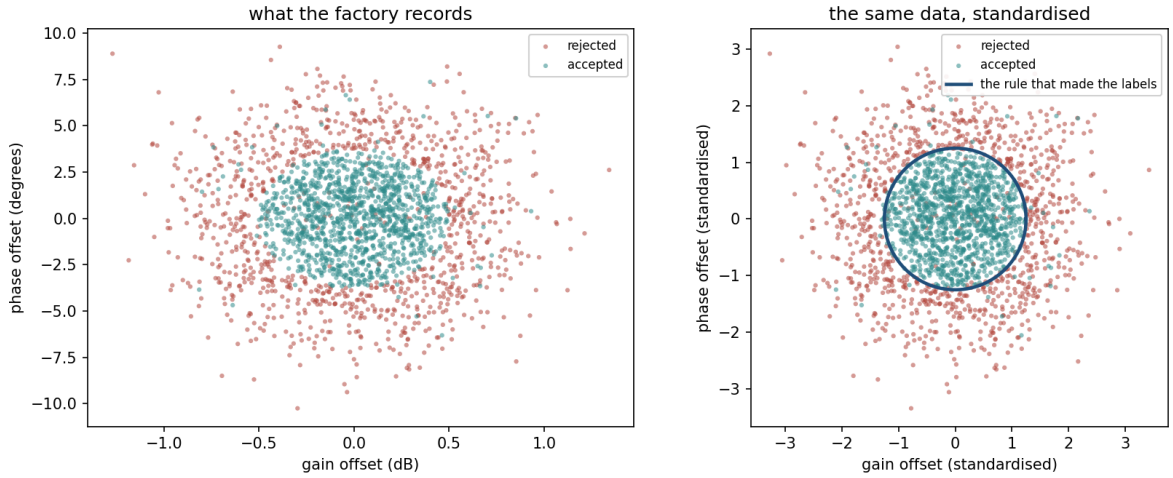
Nothing here is new. What is new is the observation that this is a *component* rather than a model — one unit, which a network stacks.

The set of points a single unit is undecided about, $\hat{y} = 0.5$, is $w^\top x + b = 0$: a straight line in two dimensions, a plane in three, a hyperplane in general. **A neuron is a line.** Everything a single neuron can express is "which side of this line are you on, and how far".



One unit drawn two ways, with $w_1 = 1$, $w_2 = 2$ and $b = -1$. Left: the inputs are weighted and added, with a constant 1 carrying the bias, and the sum z is squashed by the sigmoid; the small plot shows the sigmoid against the step the 1958 perceptron used, both switching at $z = 0$, one smoothly. Right: the same unit as a map of the input plane, coloured by $\hat{y}$. Look at the black line, $x_1 + 2x_2 - 1 = 0$: on it $z = 0$ and $\hat{y} = 0.5$, and the unit's whole verdict is which side of it a point lies on.

2.2 What one line is worth on the acceptance data



Meridian's 2,250 training sensors. Left: the raw measurements, in decibels and degrees. Right: the same points after each axis is divided by its own production spread, with the rule that generated the labels drawn on top. The accept region is a circle — and the wrong-coloured points scattered along it are the 3% of verdicts the test rig recorded incorrectly.

The right-hand panel draws something Meridian's engineers never see. The gain tolerance is 0.50 dB against a production spread of 0.40, and the phase tolerance is 3.75° against a spread of 3.00 — both exactly 1.25 spreads. So once each axis is standardised the accept region is a **circle of radius 1.25**. And because the offsets are drawn from normal distributions with those spreads, each standardised unit is a random point of the standard two-dimensional normal, so the fraction of units inside the circle — the pass rate — can be computed exactly, before any model is fitted.

For a standard two-dimensional normal the density is $\frac{1}{2\pi}e^{-(z_1^2+z_2^2)/2}$, which depends only on the distance r from the centre. In polar coordinates the area element is $r\,dr\,dt$, so integrating the angle out leaves a radial density $r\,e^{-r^2/2}$ — and that integrates in closed form, because $r\,e^{-r^2/2}$ is the derivative of $-e^{-r^2/2}$:

$$P(\|z\| < s) = \int_0^s r\,e^{-r^2/2}\,dr = 1 - e^{-s^2/2}, \qquad P(\|z\| < 1.25) = 1 - e^{-0.78125} = 0.5422$$

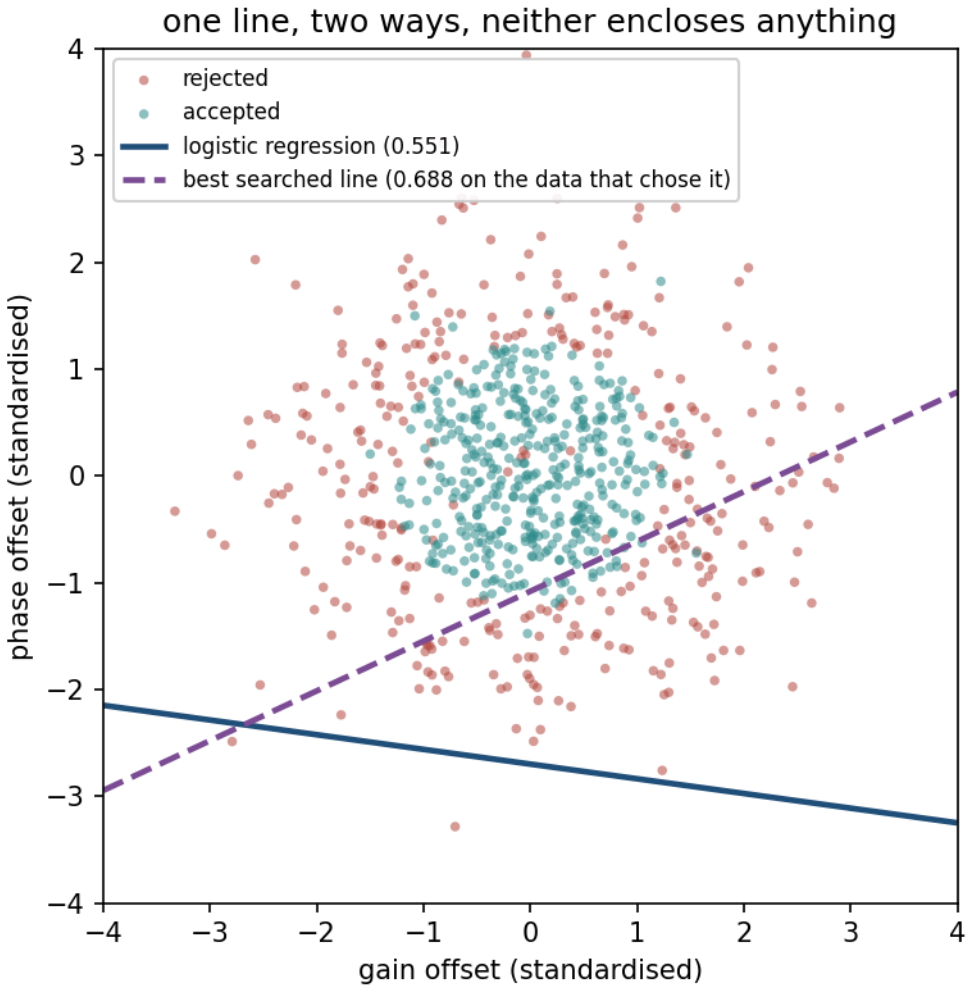
against 0.5497 measured on the 3,000 generated units. A little over half of all units pass, which is why a "model" that answers "accepted" every time, using no measurement at all, already scores about 0.55 in the table below.

Now fit logistic regression to it. The result:

	accuracy
always predict "accepted", on the 750 test units	0.5480
fitted logistic regression, on the 750 test units	**0.5507**
the best possible straight line, computed from the true distribution (section 11)	0.6491
the rig's ceiling	0.9700

The fitted coefficients are (0.0096, 0.0701) with intercept 0.1892 — almost exactly the constant model. This is not an optimiser that failed. It is the correct answer to the question a line is able to ask.

The reason is symmetry. The accept region is a disc centred on the origin, so for every accepted unit at (z_1, z_2) there is, on average, a matching one at $(-z_1, -z_2)$. Any line that gains accuracy on one side gives back the same amount on the other, and the fit settles for barely tilting at all.



The fitted boundary (solid) and the best line found by brute-force search over 72,762 candidates (dashed). Neither encloses the accept region, because no line can. The best line does not try to: it slices off one far tail, where almost every unit is a reject.

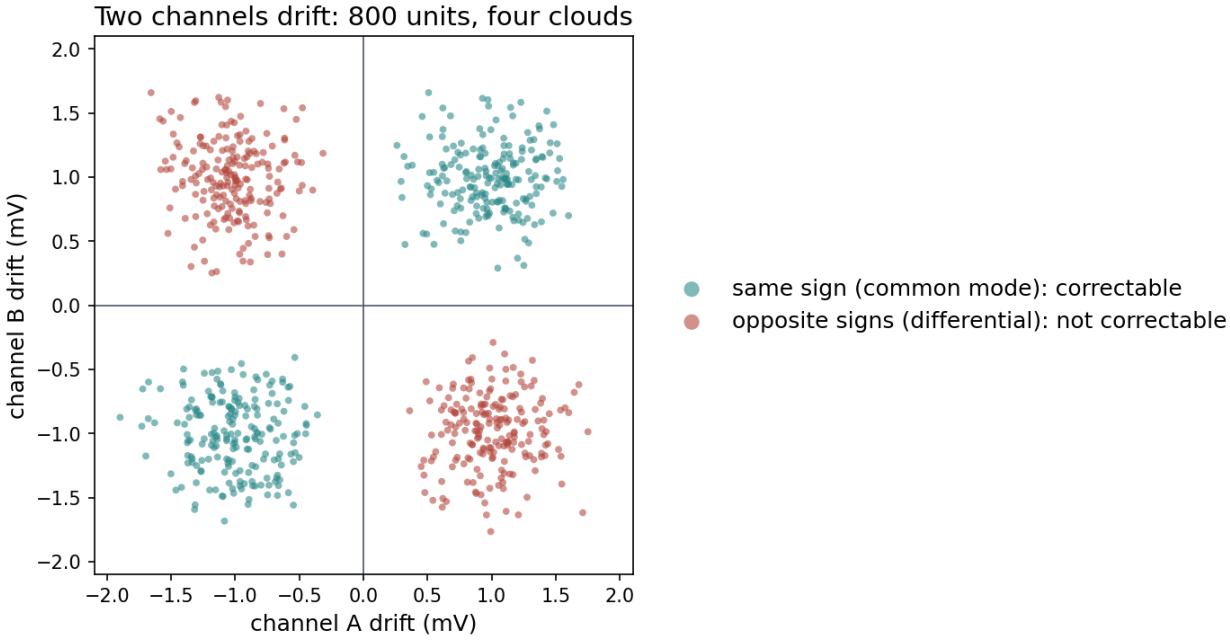
A worked note on that dashed line. Searching 72,762 candidate boundaries and reporting the winner's score *on the same data used to choose it* gives 0.6880 — an optimistic number, for the reason lesson 5 spends an hour on. The honest figure for the population, computed in section 11 without reference to any test set, is **0.6491**. Keep the difference in mind: it is 4 percentage points of pure selection bias, from a search that looks entirely innocent.

---

3. A hidden layer: fences made of lines

3.1 The smallest problem that needs one

Meridian's two-channel drift data has four clouds of 200 units each, centred at $(\pm 1, \pm 1)$ millivolts with a spread of 0.28. A unit is correctable when the two drifts share a sign. Logistic regression scores exactly **0.5000** — a line is worth nothing whatsoever.



Meridian's second problem: the drifts of a sensor's two channels, 800 units in four clouds. Teal units drifted the same way on both channels and can be corrected; rust units drifted in opposite ways and cannot. The two classes sit in opposite corners, so no single straight line can put one on each side.

Coding the two channels as ± 1, "the signs agree" means $ab = 1$, which means $a + b = \pm 2$; "the signs differ" means $a + b = 0$. So the rule is entirely about $|a + b|$:

$$\text{correctable} \iff |a + b| \text{ is large}$$

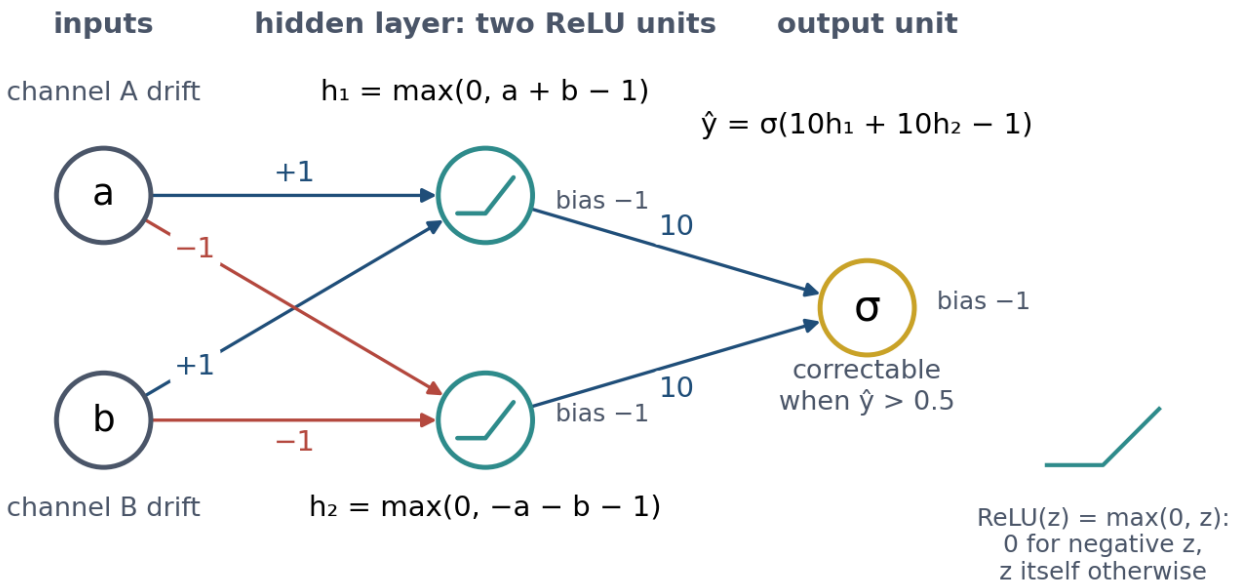
That is a one-dimensional question — and it is not one a line can answer, because "large in absolute value" is the *outside of a strip*, which needs two boundaries, not one.

3.2 A network built by hand

Two hidden units are enough, and rather than train them we can simply write them down. Give the hidden layer

$$W^{[1]} = \begin{pmatrix} 1 & -1 \\ 1 & -1 \end{pmatrix}, \quad b^{[1]} = \begin{pmatrix} -1 \\ -1 \end{pmatrix}, \quad W^{[2]} = \begin{pmatrix} 10 \\ 10 \end{pmatrix}, \quad b^{[2]} = -1$$

so that the two units compute $h_1 = \mathrm{ReLU}(a + b - 1)$ and $h_2 = \mathrm{ReLU}(-a - b - 1)$, where **ReLU** is the *rectified linear unit*, $\mathrm{ReLU}(z) = \max(0, z)$.



The same network drawn: the weights on the arrows, the biases beside the nodes, and inside each hidden node the shape of the ReLU - flat at zero for negative input, rising for positive. The first hidden unit responds only when $a + b$ is above 1, the second only when it is below -1, and the output adds the two.

Between them these two units compute $|a + b|$, clipped: h_1 is positive only when $a + b > 1$, and h_2 only when $a + b < -1$. Work the four cloud centres through by hand:

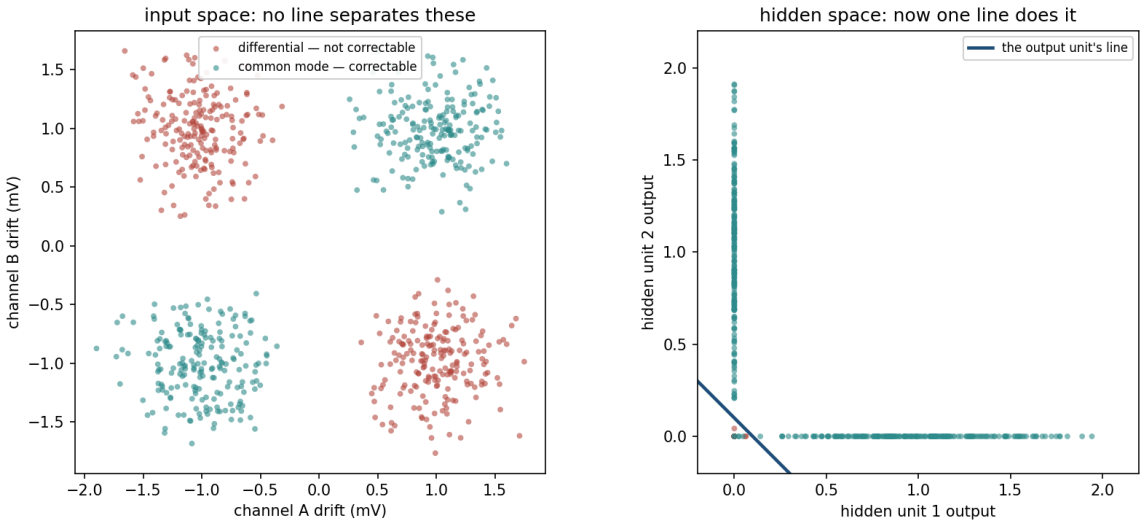
a	b	$a + b$	h_1	h_2	$10(h_1 + h_2) - 1$	$\hat{y}$	class
$+1$	$+1$	$+2$	1	0	9	0.9999	1
-1	-1	-2	0	1	9	0.9999	1
$+1$	-1	0	0	0	-1	0.2689	0
-1	$+1$	0	0	0	-1	0.2689	0

Every row is right, and the network's decision rule follows from the last column: $\hat{y} > 0.5$ exactly when $10(h_1 + h_2) - 1 > 0$, that is $h_1 + h_2 > 0.1$ — and since $h_1 + h_2 = |a + b| - 1$ whenever either unit is active, the rule is exactly $|a + b| > 1.1$. Run it over all 800 units — noise included, nothing trained — and it scores **0.9938**: five units wrong out of eight hundred.

This is worth dwelling on. The network was not fitted to anything. Its weights were chosen by reasoning about the problem, and they work. Whatever training does, what it is *searching for* is something of this kind.

3.3 What the hidden layer actually did

The hidden layer **classified nothing**. It moved the data until the last layer's line was enough. That is the whole idea, and the name for it is *representation learning*: the useful thing a network learns is not the final boundary but the coordinates in which the final boundary is simple.



Left: the four clouds in the input space, where no line separates the colours. Right: the same 800 units plotted by what the two hidden units output. The hidden layer has moved the correctable clouds to $(1, 0)$ *and* $(0, 1)$ *and stacked both uncorrectable clouds at the origin, leaving a problem the output unit's single line solves.*

Every deep architecture in the rest of the field is this observation applied repeatedly. Lesson 10's convolutional networks are the same trick with a restriction on which weights are allowed to be non-zero.

3.4 Capacity is not the same as findability

The weights above exist and score 0.9938. How often does gradient descent *find* them? Notebook 01 runs 20 restarts at each width:

hidden units	median accuracy	best of 20	runs above 0.95
2	0.7500	0.9975	**4/20**
3	1.0000	1.0000	15/20
4	1.0000	1.0000	19/20
8	1.0000	1.0000	20/20

Two units suffice and plain gradient descent finds them in one run in five. The recurring failure mode is visible in the median: 0.7500 is exactly three of the four clouds, the solution you get by using both lines to carve off a single quadrant. It is a local minimum, and from most starting points it is downhill.

Representable and findable are different properties. This is the honest reason production networks are wider than their task requires: the extra units are not extra capacity, they are extra starting points, so that some unit begins near a useful line.

3.5 Universal approximation, and what it does not promise

There is a theorem here, and it is more often cited than read. In the form due to Cybenko (1989) and Hornik (1991): a network with **one** hidden layer, a non-polynomial activation, and enough

units can approximate any continuous function on a bounded region to any accuracy you like.

Read the quantifiers carefully, because the theorem promises far less than its reputation suggests. It says such a network *exists*. It says nothing about how many units "enough" is, nothing about whether any training procedure will find it, and nothing about how the network behaves outside the region. Section 3.4 is a two-unit counterexample to the reading people usually take away: the approximation existed, and gradient descent missed it 16 times in 20.

The theorem's real content is a licence to stop worrying about expressiveness and start worrying about optimisation and data — which is what the rest of this lesson does.

---

4. Forward propagation

4.1 Notation, and why the shapes matter

One hidden layer of H units and one output unit, for m examples and n inputs:

$$Z^{[1]} = XW^{[1]} + b^{[1]}, \qquad A^{[1]} = g(Z^{[1]})$$

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}, \qquad \hat{y} = \sigma(Z^{[2]})$$

with shapes

symbol	shape	what it is
X	$m \times n$	the design matrix, one example per row
$W^{[1]}$	$n \times H$	one column per hidden unit
$b^{[1]}$	H	broadcast across rows
$Z^{[1]}, A^{[1]}$	$m \times H$	pre-activations and activations
$W^{[2]}$	$H \times 1$	the output unit's weights
$\hat{y}$	$m \times 1$	one prediction per example

This course puts examples in **rows**, which is what scikit-learn and Keras both do. Many textbooks put them in columns, and every transpose in the derivation below flips if you do. Mixing the two conventions halfway through a derivation is the single most common way to produce algebra that looks right and is not.

Two structural facts follow from the shapes, and both are worth stating.

Examples never mix. The row index i passes through every operation untouched: nothing in the forward pass lets example 3 influence example 7. That is what makes mini-batching valid.

Column j of $W^{[1]}$ is hidden unit j. It is the unit's weight vector, and the line $W^{[1]}_{:,j} \cdot z + b^{[1]}_j = 0$ is the line that unit draws. This is how the left panel of the fence figure in section 11 is plotted — straight from the weight matrix.

The cost, for binary classification, is the cross-entropy of lesson 4:

$$J = -\frac{1}{m}\sum_{i=1}^{m}\Big[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\Big]$$

An untrained network on the acceptance data costs 0.6721, against $\log 2 = 0.6931$ for guessing — it starts, as it should, knowing essentially nothing.

---

5. Backpropagation

5.1 The idea, before any algebra

We need $\partial J/\partial\theta$ for every weight and bias. There are two obvious ways to get it and both are bad.

Perturbing each parameter and re-running the network costs one forward pass per parameter. For the 301,066-parameter network of section 10 that is 301,066 forward passes for a single step.

Differentiating the whole composed expression symbolically produces something that repeats the same sub-expressions thousands of times.

Backpropagation is the observation that those repeated sub-expressions can be computed **once each, right to left**. The quantity worth carrying is $\partial J/\partial Z^{[l]}$ — how much the cost changes per unit change in layer l's pre-activations. Given it for layer l, you get layer l's weight gradients immediately, *and* the same quantity for layer $l-1$. So one backward sweep produces every gradient, at about the cost of one forward pass.

The intuition in words: **a weight's gradient is how wrong the layer above it was, multiplied by how much this weight contributed to that layer's input.** Everything below is that sentence in matrix form.

5.2 The output layer, and why sigmoid and cross-entropy belong together

Start at the end. For a single example, with $z = Z_i^{[2]}$ and $\hat{y} = \sigma(z)$:

$$\frac{\partial J_i}{\partial \hat{y}} = -\frac{1}{m}\left[\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right] = \frac{1}{m}\cdot\frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$$

The sigmoid's derivative is

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)(1-\sigma(z)) = \hat{y}(1-\hat{y})$$

and the chain rule multiplies the two:

$$\frac{\partial J_i}{\partial z} = \frac{1}{m}\cdot\frac{\hat{y}-y}{\hat{y}(1-\hat{y})}\cdot\hat{y}(1-\hat{y}) = \frac{1}{m}(\hat{y}-y)$$

The denominator cancels exactly. Writing $\delta^{[2]} = \partial J/\partial Z^{[2]}$, an $m \times 1$ array:

$$\boxed{\delta^{[2]} = \tfrac{1}{m}(\hat{y}-y)}$$

This cancellation is why the pair is used. With squared error instead, the $\hat{y}(1-\hat{y})$ factor survives, and it is near zero whenever the network is confidently wrong — precisely the case where you most want a large gradient. Cross-entropy removes the factor and the confidently-wrong example gets a gradient proportional to how wrong it is.

5.3 Weights, biases, and the step to the layer below

With $Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}$, in components $z_i = \sum_k A^{[1]}_{ik} W^{[2]}_k + b^{[2]}$. Differentiating,

$$\frac{\partial J}{\partial W^{[2]}_k} = \sum_{i=1}^{m} \delta^{[2]}_i A^{[1]}_{ik} \qquad \Longrightarrow \qquad \frac{\partial J}{\partial W^{[2]}} = \left(A^{[1]}\right)^\top \delta^{[2]}$$

which is $(H \times m)(m \times 1) = H \times 1$ — the shape of $W^{[2]}$, as it must be. The bias appears once per example, so

$$\frac{\partial J}{\partial b^{[2]}} = \sum_{i=1}^{m} \delta^{[2]}_i$$

Now step down. $A^{[1]}_{ik}$ affects the cost only through z_i, with $\partial z_i / \partial A^{[1]}_{ik} = W^{[2]}_k$, so

$$\frac{\partial J}{\partial A^{[1]}} = \delta^{[2]} \left(W^{[2]}\right)^\top \qquad (m \times H)$$

and passing back through the activation, which acts element by element,

$$\boxed{\delta^{[1]} = \frac{\partial J}{\partial Z^{[1]}} = \left(\delta^{[2]} \left(W^{[2]}\right)^\top\right) \odot g'(Z^{[1]})}$$

with $\odot$ the elementwise product. Then, identically to the layer above,

$$\frac{\partial J}{\partial W^{[1]}} = X^\top \delta^{[1]} \quad (n \times H), \qquad \frac{\partial J}{\partial b^{[1]}} = \sum_{i=1}^{m} \delta^{[1]}_{i\cdot} \quad (H)$$

For the ReLU the derivative is an indicator:

$$g'(z) = \mathbb{1}[z > 0]$$

so a unit whose pre-activation was negative passes **nothing** backward — which is right, because it contributed nothing forward. (At $z = 0$ the derivative does not exist; every implementation picks 0 or 1 and the choice has no measurable effect, since exact zeros essentially never occur.)

5.4 The algorithm, and what it costs

Forward. $Z^{[1]} = XW^{[1]} + b^{[1]}$; $A^{[1]} = g(Z^{[1]})$; $Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}$; $\hat{y} = \sigma(Z^{[2]})$. Keep $Z^{[1]}$ and $A^{[1]}$.

Backward. $\delta^{[2]} = \frac{1}{m}(\hat{y} - y)$; $\nabla W^{[2]} = (A^{[1]})^\top \delta^{[2]}$; $\delta^{[1]} = (\delta^{[2]}(W^{[2]})^\top) \odot g'(Z^{[1]})$; $\nabla W^{[1]} = X^\top \delta^{[1]}$.

Update. $\theta \leftarrow \theta - \alpha \nabla_\theta J$.

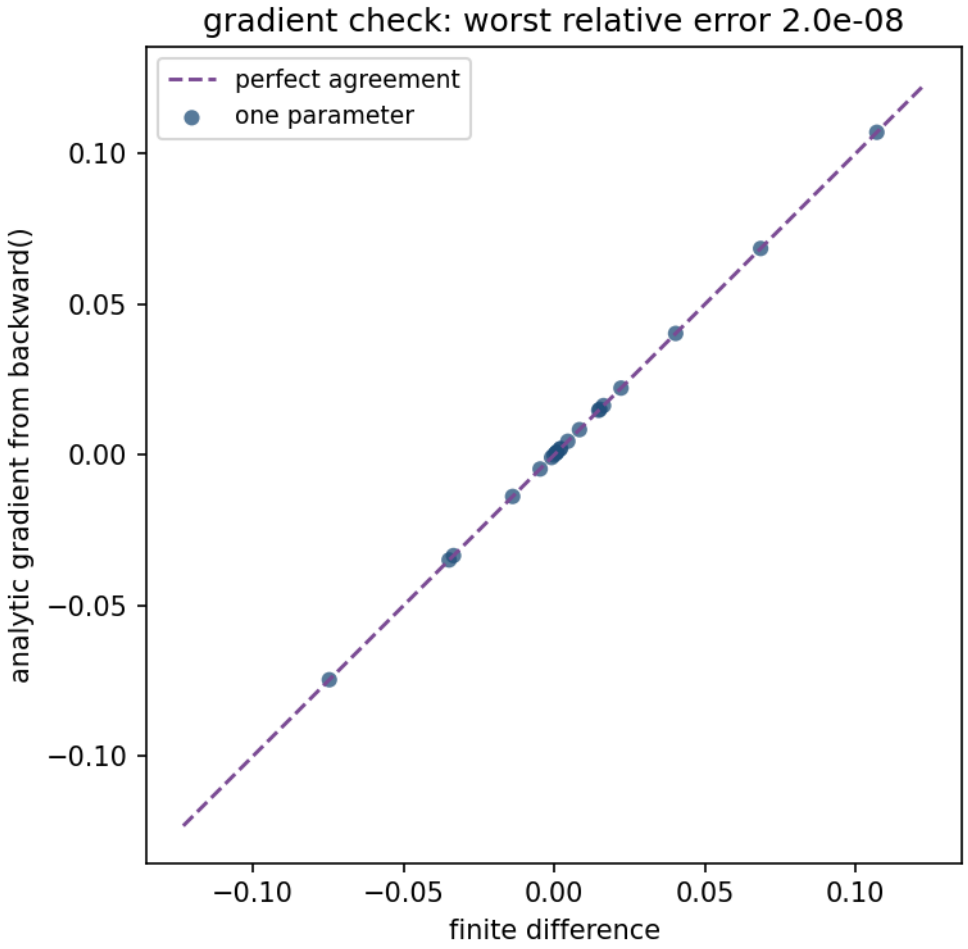
Two costs are worth naming. The backward pass does the same number of multiply-accumulate operations as the forward pass, to within a factor of about two — so a gradient costs roughly what a prediction costs, which is the fact that makes training feasible at all. And it requires **storing** every $A^{[l]}$ from the forward pass, so memory grows with depth times batch size. That memory, not arithmetic, is what usually limits batch size in practice.

5.5 Check the gradient before trusting it

A wrong analytic gradient does not raise an exception. It trains — badly, slowly, or to the wrong place — and looks like a modelling problem. The defence is four lines and it is not optional:

$$\frac{\partial J}{\partial \theta_i} \approx \frac{J(\theta + he_i) - J(\theta - he_i)}{2h}$$

Why the two-sided version: expand both terms as Taylor series, $J(\theta \pm he_i) = J \pm hJ' + \frac{h^2}{2}J'' \pm \frac{h^3}{6}J''' + ...$ Subtracting, the even-order terms cancel, $J(\theta + he_i) - J(\theta - he_i) = 2hJ' + \frac{h^3}{3}J''' + ...$, and dividing by $2h$ leaves $J' + \frac{h^2}{6}J'''$: an error of order h^2. The one-sided $\big(J(\theta + he_i) - J(\theta)\big)/h$ keeps the $\frac{h}{2}J''$ term, an error of order h. With $h = 10^{-5}$ that is 10^{-10} against 10^{-5}, and in double precision the 10^{-10} is already down at the level of rounding.



Every one of the 21 partial derivatives of a small network, analytic against finite-difference. The points lie on the diagonal to eight decimal places.

Notebook 01 measures a worst relative disagreement of 1.97×10^{-8} and a median of 7.83×10^{-11}. **Below about 10^{-6}, believe the gradient; above about 10^{-4}, there is a bug.**

Run this once, on a small network with a handful of examples, whenever you write a backward pass by hand. It is the single highest-value four lines in this lesson.

---

6. More than two classes

6.1 Softmax

Ten classes need ten output units and a way to turn ten scores into a probability distribution:

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Positive by construction, summing to one, and monotone in each z_k. For $K = 2$ it reduces to the

sigmoid: dividing top and bottom by e^{z_1}, $e^{z_1}/(e^{z_1}+e^{z_2}) = 1/(1+e^{-(z_1-z_2)}) = \sigma(z_1 - z_2)$ — only the difference of the two scores matters, exactly as in lesson 6's Naive Bayes normalisation. The loss is categorical cross-entropy, which for a one-hot y is just $-\log \hat{y}_c$ for the true class c.

In practice the exponentials are computed as $e^{z_k - \max_j z_j}$, which changes nothing mathematically — the constant cancels between numerator and denominator — and prevents an overflow that is otherwise easy to hit.

6.2 The same cancellation, one dimension up

The softmax Jacobian follows from the quotient rule. Write $\hat{y}_k = e^{z_k}/S$ with $S = \sum_j e^{z_j}$, and note $\partial S/\partial z_l = e^{z_l}$:

$$\frac{\partial \hat{y}_k}{\partial z_l} = \frac{\delta_{kl}\, e^{z_k} S - e^{z_k} e^{z_l}}{S^2} = \hat{y}_k(\delta_{kl} - \hat{y}_l)$$

where δ_{kl} is 1 if $k = l$ and 0 otherwise. With $L = -\sum_k y_k \log \hat{y}_k$,

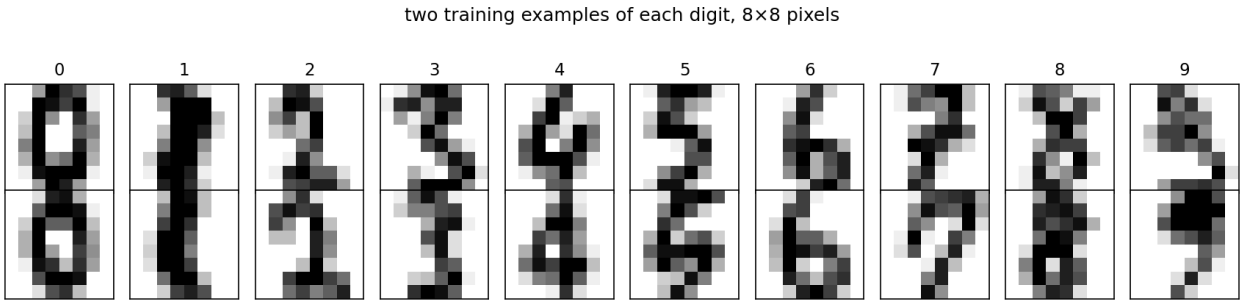
$$\frac{\partial L}{\partial z_l} = -\sum_k \frac{y_k}{\hat{y}_k}\, \hat{y}_k(\delta_{kl} - \hat{y}_l) = -\sum_k y_k(\delta_{kl} - \hat{y}_l) = -y_l + \hat{y}_l \sum_k y_k = \hat{y}_l - y_l$$

using $\sum_k y_k = 1$. **The same result as the binary case**: the gradient at the output layer is prediction minus truth, and everything in section 5.3 applies unchanged with $\delta^{[2]}$ now $m \times K$.

That this holds for both is not a coincidence. Sigmoid-with-cross-entropy and softmax-with-cross-entropy are the same construction for $K = 2$ and general K, and in both cases the loss is chosen as the one whose derivative cancels the output non-linearity's.

6.3 Does a hidden layer help on digits?

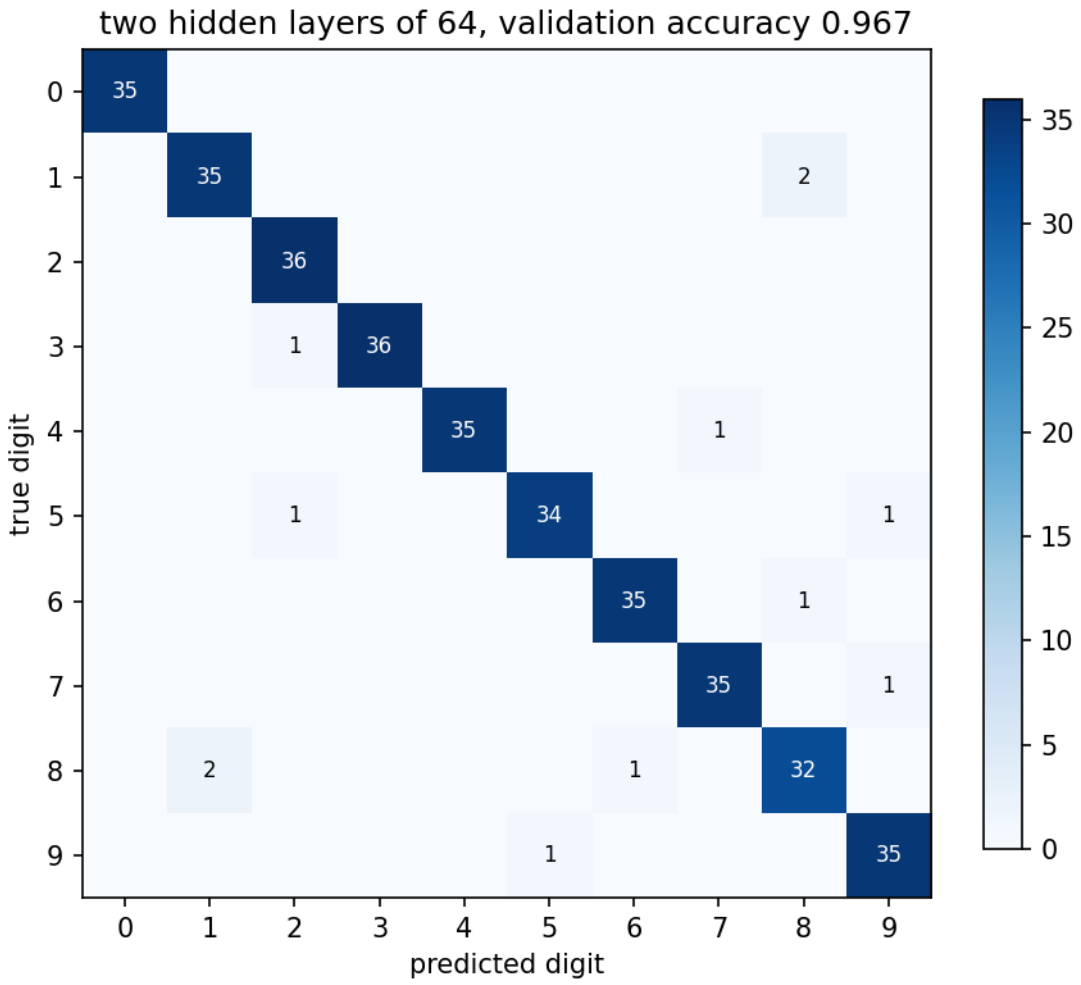
architecture	parameters	validation accuracy	sd
softmax alone, no hidden layer	650	0.9324	0.0035
one hidden layer of 32	2,410	0.9602	0.0052
one hidden layer of 64	4,810	0.9611	0.0045
two hidden layers of 64	8,970	0.9676	0.0057



Two training examples of each digit. At 8×8, several are ambiguous to a human reader — this dataset has a ceiling too, and unlike the acceptance data nobody has published what it is.

Multiclass logistic regression, with no hidden layer at all, is within 3.5 points of the best network on the table. The first hidden layer is worth about 2.8 points; **doubling it from 32 units to 64 is worth 0.09 points**, which is a fifth of the seed-to-seed spread and therefore nothing.

Compare the acceptance problem, where the same step was worth 39 points. The difference is structural: the digits are close to linearly separable in pixel space, because classes differ in *which pixels are dark* and a weighted sum of pixels captures most of that. The acceptance rule depended on two measurements *in combination*, which is exactly what a weighted sum cannot express.



Validation confusion matrix for the two-layer network. The errors are few and unsurprising — a 1 called an 8 twice, an 8 called a 1 twice — which is what being near a dataset's ceiling looks like.

---

7. Activations, and the vanishing gradient

7.1 Why a non-linearity is needed at all

Suppose g were the identity. Then

$$Z^{[2]} = (XW^{[1]} + b^{[1]})W^{[2]} + b^{[2]} = X(W^{[1]}W^{[2]}) + (b^{[1]}W^{[2]} + b^{[2]})$$

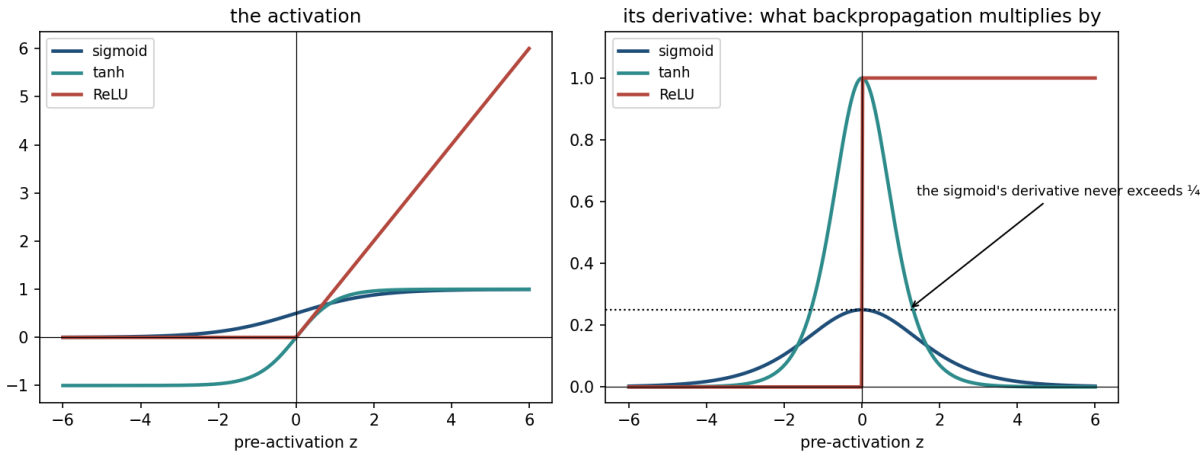
which is a single linear layer with weights $W^{[1]}W^{[2]}$. Any number of linear layers composes to one. **The non-linearity is the entire reason depth buys anything**, and this collapse is worth remembering, because a network whose activations have all saturated or all died has effectively performed it.

7.2 The sigmoid's derivative is bounded by one quarter

From section 5.2, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Writing $s = \sigma(z) \in (0, 1)$, the function $s(1 - s)$ is a downward parabola with its maximum at $s = \frac{1}{2}$, giving

$$\max_z \sigma'(z) = \tfrac{1}{2} \cdot \tfrac{1}{2} = \tfrac{1}{4}$$

attained at $z = 0$ and nowhere else.



Left: three activations. Right: their derivatives — the quantity backpropagation multiplies by once per layer. The sigmoid's never exceeds ¼, and that horizontal line is the subject of this section.
Now look again at the backward recursion:

$$\delta^{[l]} = \left(\delta^{[l+1]}(W^{[l+1]})^\top\right) \odot g'(Z^{[l]})$$

Two factors act on the gradient at each layer: the weight matrix, and the activation's derivative. Initialisation (section 8.2) is chosen precisely so that the weight matrix contributes a factor of about 1. That leaves the derivative in charge, and it can only shrink:

$$\|\delta^{[l]}\| \lesssim \tfrac{1}{4}\|\delta^{[l+1]}\|$$

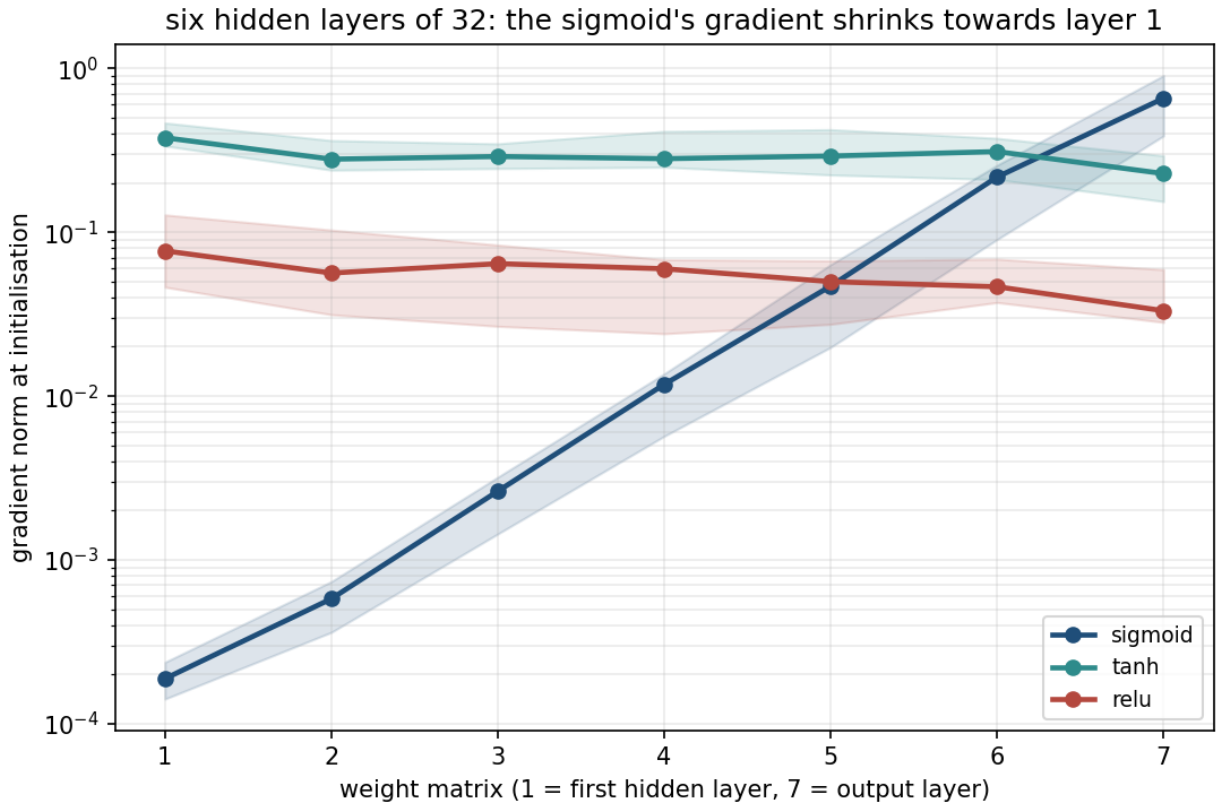
Over L layers this compounds to a factor of roughly 4^L. At initialisation the bound is close to tight, not loose, because the pre-activations sit near zero — exactly where σ' takes its maximum.

This is the one number to carry out of the lesson: a sigmoid layer divides the gradient by about four.

7.3 What that costs, measured

Notebook 02 measures the per-layer shrinkage across eight seeds:

$$3.90,\ 3.97,\ 4.14,\ 4.05,\ 3.92,\ 4.31,\ 4.06,\ 3.86$$



Gradient norms at every weight matrix of an untrained six-hidden-layer network, on a logarithmic scale; the band spans eight random initialisations. The sigmoid falls by three and a half orders of magnitude from output to input. tanh and the ReLU are flat.

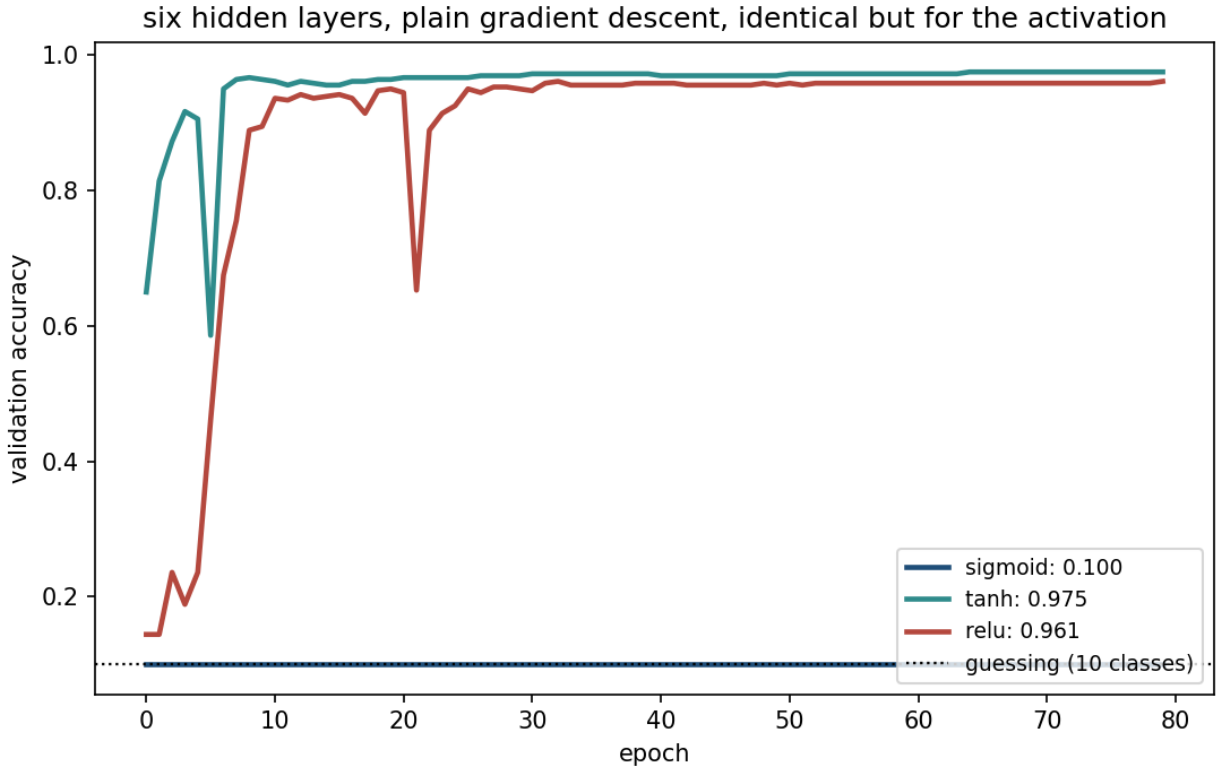
All eight land between 3.9 and 4.3 — scattered around 4, as predicted. The end-to-end ratio between the last weight matrix and the first has a median of **3,547**, and this is where a caution belongs. That figure ranges from 2,734 to 4,607 across the same eight seeds. Quoting the largest as "the gradient shrinks 4,607-fold" would be reporting one draw as though it were a law. The per-layer factor is the property; the end-to-end number is what it compounds to, and it inherits six layers' worth of scatter.

Varying depth confirms the mechanism rather than just the number:

hidden layers	median ratio	lowest	highest	4^{depth}
1	2.3	1.9	2.3	4
2	9.5	7.4	13.4	16
4	169.6	131.4	196.5	256
6	3,552.9	2,740.8	4,606.8	4,096

hidden layers	median ratio	lowest	highest	4^{depth}
8	46,250.0	41,174.1	76,144.0	65,536

Geometric growth over four orders of magnitude, staying within a factor of two of 4^{depth} at every depth and always slightly below it — the weight matrices give back a little of what the derivative takes.



The same six-layer architecture trained three times, differing only in the activation. The sigmoid never leaves chance in eighty epochs. Both others train within a handful.

The consequence is not subtle. The sigmoid network sits at **0.1000** — exact chance on ten classes — after eighty epochs. tanh reaches 0.85 in 3 epochs and finishes at 0.9750; the ReLU takes 9 epochs and finishes at 0.9611.

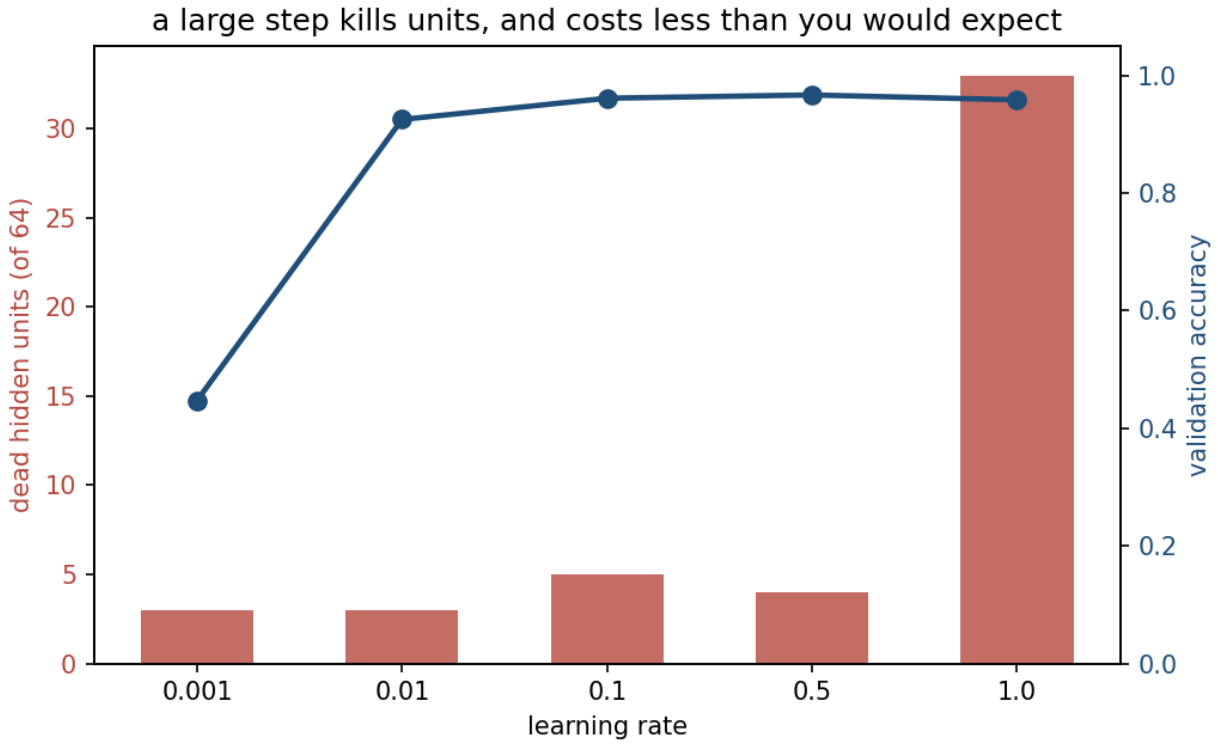
Note which one won, because it is not the one the slogan predicts. **tanh beat the ReLU here**, and that is consistent with the argument rather than a surprise: tanh is every bit as much a squashing function as the sigmoid, and it is fine, because its derivative peaks at 1 rather than ¼. What kills the sigmoid is not squashing. It is *where its derivative is bounded.*

"Use ReLU" remains good default advice — it is cheaper to compute, and it does not saturate for large positive input — but on six layers of 32 units those advantages do not show up, and pretending they did would be teaching a slogan instead of a mechanism.

7.4 The predictable mistake: the ReLU has its own failure

Having met the vanishing gradient, most people conclude the ReLU has no gradient problem. The reasoning is sound as far as it goes — its derivative really is exactly 1 on the active side, and it really does not saturate.

But a ReLU unit whose pre-activation is negative for *every* training example outputs zero for all of them, and therefore receives zero gradient from all of them, for ever. It is **dead**, and no amount of further training revives it: the gradient that would move it is the one it cannot receive. A single large step is enough to knock a bias far enough negative to cause this.



Dead units in a 64-unit layer after 40 epochs, against learning rate, with validation accuracy on the right axis. At $\alpha = 1.0$, 33 of 64 units are dead — and accuracy is 0.9583 against a best of 0.9667.

The measured cost is smaller than the word "dead" suggests: with half the layer dead the survivors absorb the work and accuracy drops by less than one point. So dead units are usually **wasted capacity rather than catastrophe**, which is exactly why the failure is easy to miss — nothing in the training curve announces it. A layer of 64 that is half dead is a layer of 31, and if that layer were your bottleneck you would be tuning everything except the thing that is wrong.

---

8. Initialisation

8.1 Zero cannot work, and the reason is not what it first appears

Lesson 3 started gradient descent from $w = 0$ and that was correct: for linear and logistic regression the cost is convex and the origin is as good a start as any. Carrying the habit into a network

destroys it.

The usual explanation is symmetry: if every hidden unit starts identical, then every unit computes the same output, receives the same gradient, takes the same step, and stays identical for ever. That argument is right, and it applies to any initialisation that gives two units the same weights.

For **all** weights zero, something stronger and more specific happens. Since $W^{[2]} = 0$,

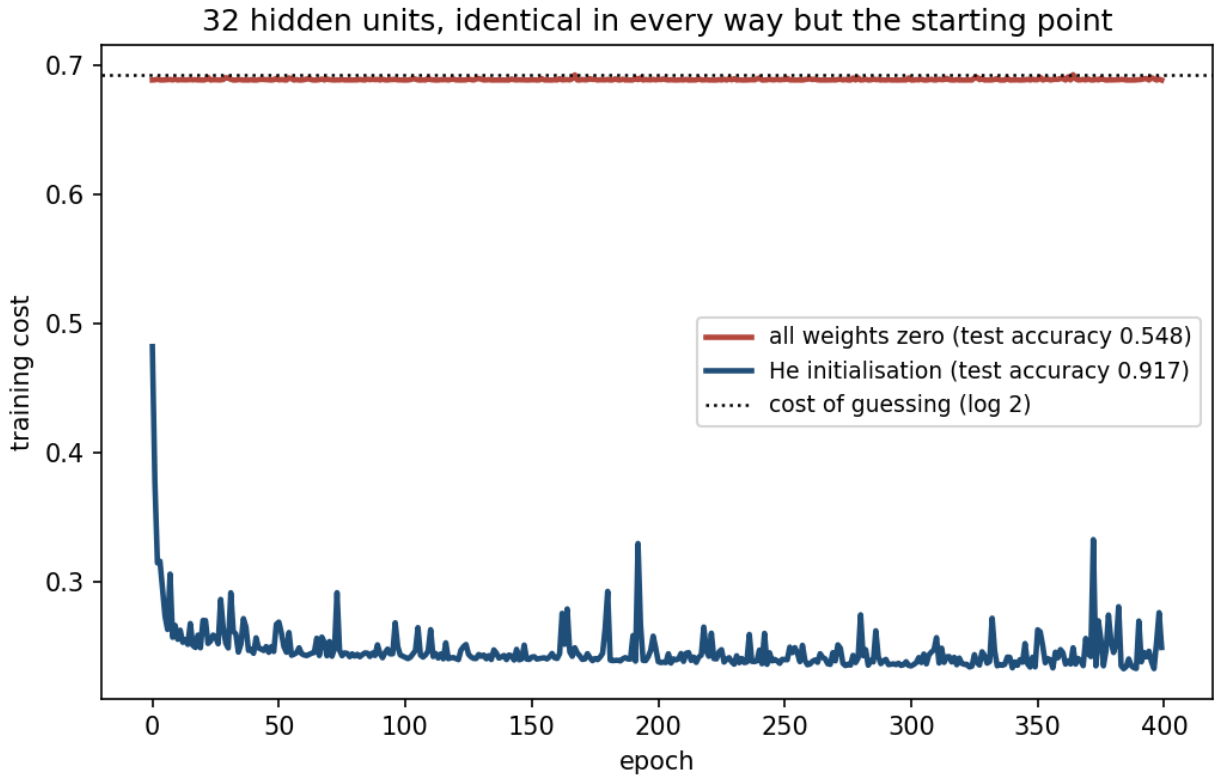
$$\delta^{[1]} = \left(\delta^{[2]}(W^{[2]})^\top\right) \odot g'(Z^{[1]}) = 0$$

so $W^{[1]}$ and $b^{[1]}$ receive exactly zero gradient. And since $A^{[1]} = \mathrm{ReLU}(0) = 0$, the gradient $\nabla W^{[2]} = (A^{[1]})^\top \delta^{[2]}$ is zero too. **Every parameter in the network is frozen except $b^{[2]}$.**

The network is therefore a constant predictor, and $b^{[2]}$ converges to the value making that constant the training base rate $\bar{y}$. Its cost converges to the entropy of the label distribution:

$$J \to -[\bar{y}\log\bar{y} + (1-\bar{y})\log(1-\bar{y})]$$

With $\bar{y} = 0.5471$ on the training set this is **0.6887**, and notebook 01 measures the final cost as 0.6887. Not approximately — the prediction is exact, because the mechanism is exact.



32 hidden units, identical in data, architecture and learning rate, differing only in the starting point. The zero-initialised network flattens at the entropy of the label distribution; the randomly initialised one trains.

Counting distinct columns of $W^{[1]}$ afterwards: **1 of 32** from the zero start, 32 of 32 from the random one. Random initialisation is not a heuristic that happens to help — it is what makes the units *different problems to solve*.

8.2 How large? Propagating the variance

Zero is excluded, but scale is still free, and it is not a free parameter either. A unit computes $z = \sum_{i=1}^{n} w_i a_i$ over n inputs. Taking the w_i independent of the a_i, mutually independent, and zero-mean, that sum's variance follows in two steps.

Independent terms add their variances, so the sum of n identically distributed products contributes n copies of one product's variance:

$$\mathrm{Var}(z) = \sum_{i=1}^{n} \mathrm{Var}(w_i a_i) = n\,\mathrm{Var}(wa)$$

And one product's variance is a product of expectations, because w has mean zero and is independent of a: $\mathbb{E}[wa] = \mathbb{E}[w]\mathbb{E}[a] = 0$, so the squared mean drops out of $\mathrm{Var}(wa) = \mathbb{E}[w^2a^2] - \mathbb{E}[wa]^2$ and independence splits what is left:

$$\mathrm{Var}(wa) = \mathbb{E}[w^2]\,\mathbb{E}[a^2] = \mathrm{Var}(w)\,\mathbb{E}[a^2] \qquad \Longrightarrow \qquad \mathrm{Var}(z) = n\,\mathrm{Var}(w)\,\mathbb{E}[a^2]$$

using $\mathbb{E}[w^2] = \mathrm{Var}(w)$ for the same zero-mean reason.

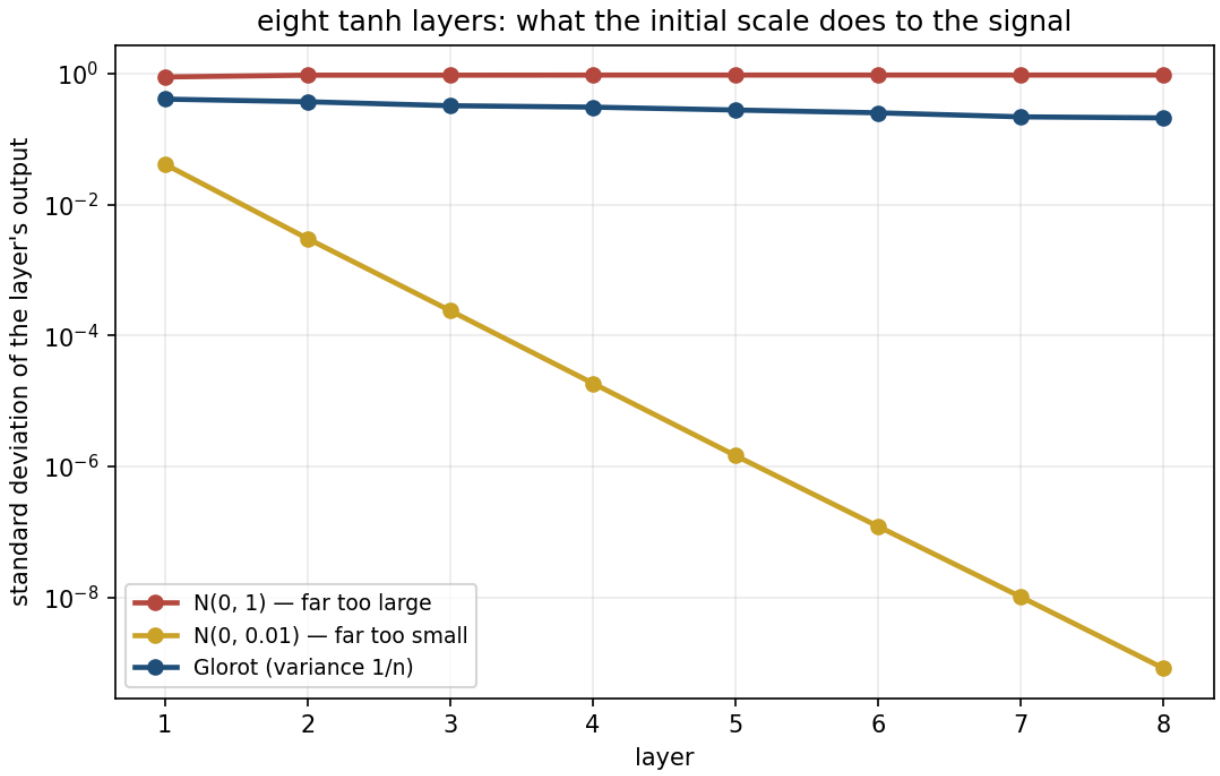
If the incoming activations are also zero-mean, $\mathbb{E}[a^2] = \mathrm{Var}(a)$ and the variance is multiplied by $n\,\mathrm{Var}(w)$ at every layer. Anything other than 1 compounds geometrically with depth. So

$$\mathrm{Var}(w) = \frac{1}{n} \qquad \text{(Glorot / Xavier)}$$

For the ReLU the derivation changes in one place. It zeroes the negative half, so a^2 equals z^2 exactly when $z > 0$ and is 0 otherwise — and for a symmetric z that is half the draws, so $\mathbb{E}[a^2] = \frac{1}{2}\mathbb{E}[z^2] = \frac{1}{2}\mathrm{Var}(z)$: half the variance survives each layer. Compensating gives

$$\mathrm{Var}(w) = \frac{2}{n} \qquad \text{(He)}$$

which is what `initialise` uses in notebook 01, and what `kernel_initializer="he_normal"` means in Keras.



The standard deviation of each layer's output in an eight-layer tanh network, for three initialisation scales. Too small collapses by an order of magnitude per layer; too large saturates; Glorot roughly holds.

layer	$\mathcal{N}(0,1)$	$\mathcal{N}(0,0.01)$	Glorot
1	0.8858	0.0412	0.4063
4	0.9463	0.0000	0.3066
8	0.9472	0.0000	0.2102

The two failures are different and both are fatal. **Too small** collapses: by layer 4 the signal is indistinguishable from zero, and nothing downstream can recover what is no longer there. **Too large** does not explode — tanh cannot exceed 1 — it *saturates*: 73% of the last layer's units sit past $|a| > 0.99$, where the derivative is indistinguishable from zero. A saturated layer passes signal forward and nothing backward, which is section 7's failure arriving from the other direction.

Glorot loses about half its spread over eight layers. That is what "preserving the signal" looks like in practice, against three orders of magnitude and near-total saturation for the alternatives.

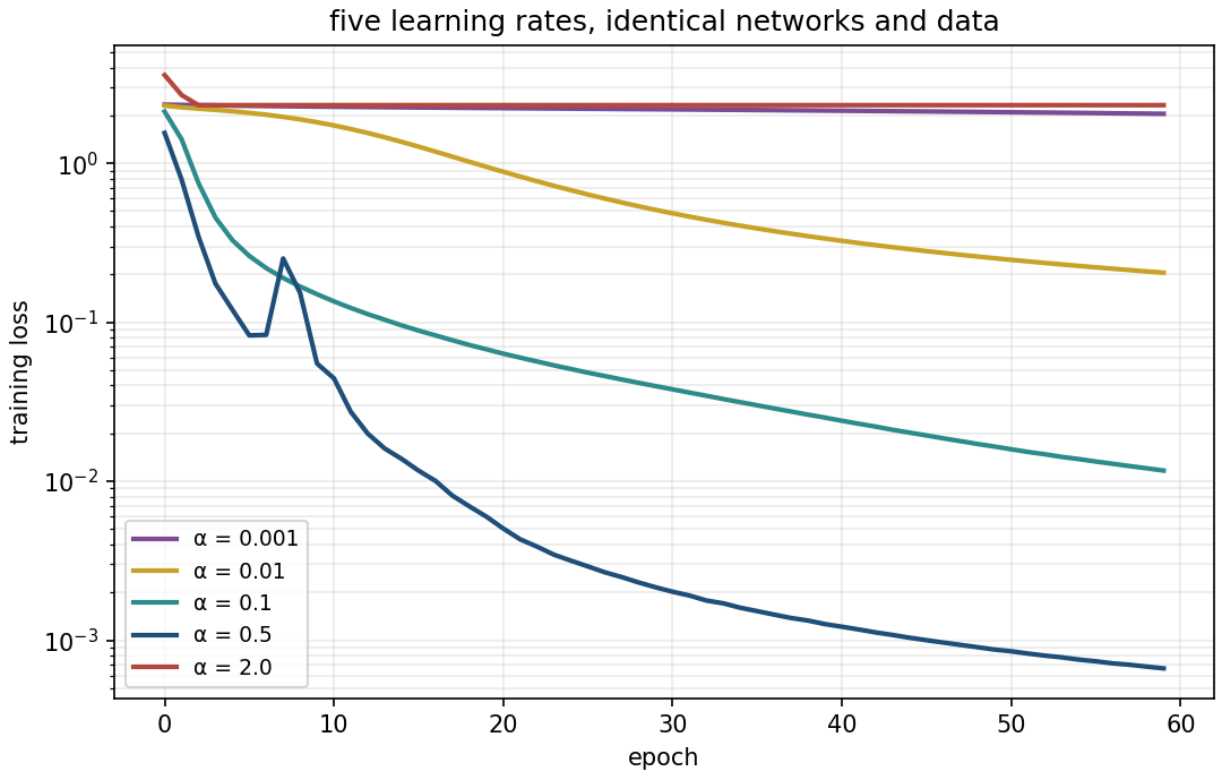
---

9. Optimisation in practice

9.1 The learning rate

α is the first thing to sweep and the last thing to guess. Notebook 03 runs the same network at five rates:

α	final training loss	validation accuracy	sd
0.001	1.9550	0.5574	0.0794
0.01	0.1965	0.9398	0.0035
0.1	0.0115	0.9556	0.0039
0.5	0.0007	**0.9685**	0.0026
2.0	2.3222	0.1009	0.0013



Training loss for five learning rates on identical networks and data, logarithmic vertical axis. Between "still falling when the epochs ran out" and "never falls at all" there is about one and a half orders of magnitude of useful range.

Three regimes, and the useful band between them is narrow. Too large and the steps overshoot every minimum they approach: 0.1009 is exact chance on ten classes. Too small and the loss is still falling when the epochs run out.

The predictable mistake is diagnosing the second case as a capacity problem. A network underfitting because α is too small looks exactly like a network that is too small — training and validation accuracy both low, both still improving — and the reasonable response, adding units and layers, makes it slower without making it better. The reasoning is sound: low training accuracy really is the classic signature of underfitting. It is just that the cause is in the optimiser, not the architecture. **Vary the learning rate over orders of magnitude before touching the architecture.**

Notebook 01 has a second instance of the same failure. At $\alpha = 0.5$ its four-unit network reaches a training cost of 0.2564 at epoch 268 and then *climbs* to 0.8737 by epoch 400: five runs in eight end

above their own minimum. A rate that is stable early can be unstable later, once the network is in a sharper part of the cost surface.

9.2 Mini-batches

Full-batch gradient descent computes ∇J on all m examples per step. Stochastic gradient descent (SGD) uses one. Mini-batch SGD, which is what everyone means by SGD in practice, uses a few dozen to a few hundred, and sits between the two: its gradient estimate is noisy but right on average (unbiased), a batch is computed as one matrix product, which hardware does fast, and the noise itself helps escape the shallow local minima of section 3.4.

Everything in this lesson uses batches of 32 or 64. The learning rate and the batch size interact — a larger batch gives a less noisy gradient and tolerates a larger α — so they should not be tuned independently.

9.3 Momentum and Adam

Momentum accumulates a running average of past gradients:

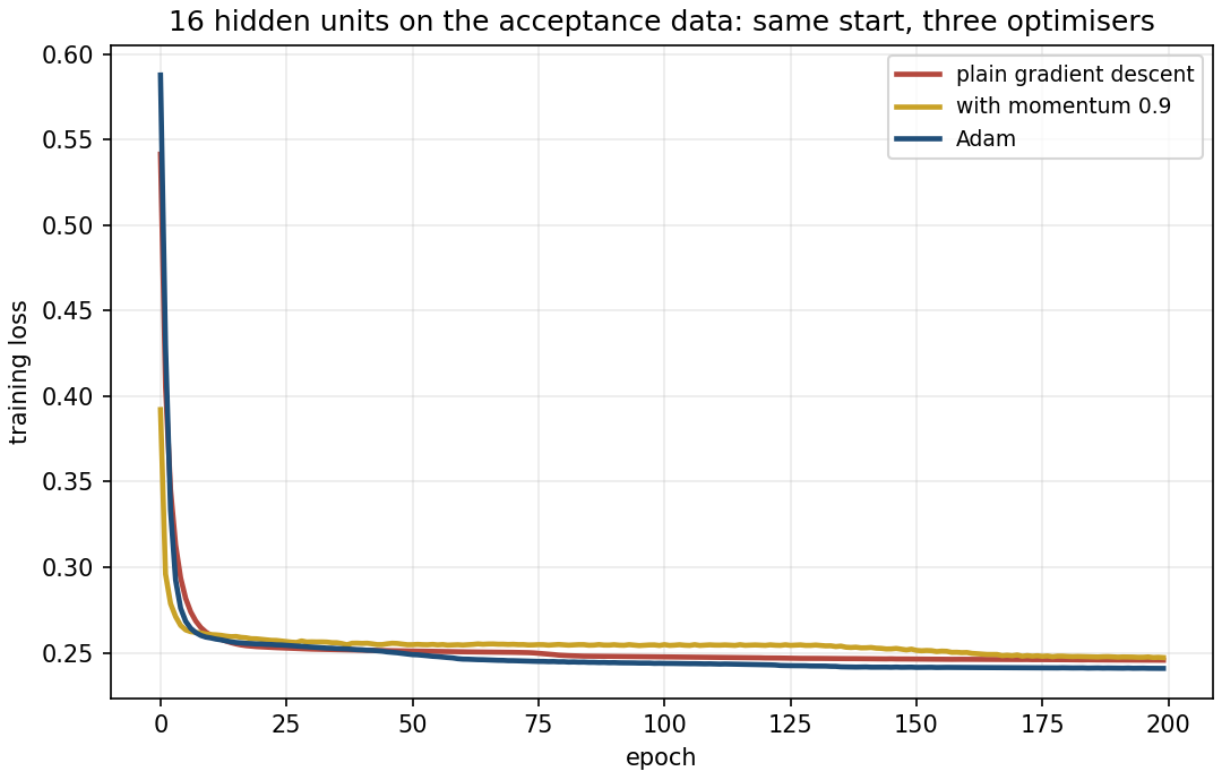
$$v \leftarrow \beta v + (1-\beta)\nabla J, \qquad \theta \leftarrow \theta - \alpha v$$

with $\beta = 0.9$ typically. Consistent directions accumulate; directions that oscillate cancel. This is the direct repair for the stretched-valley problem lesson 3 described through the condition number.

Adam (adaptive moment estimation) adds a per-parameter step size, dividing each coordinate's step by a running estimate of that coordinate's own gradient magnitude. That is what makes it forgiving of a badly chosen global α, and it is the reason it is the default almost everywhere.

Notebook 03 puts all three on the acceptance problem, at the width where notebook 01's plain gradient descent had stalled:

optimiser	test accuracy	sd	worst of 5	vs the true rule
plain gradient descent	0.9389	0.0184	0.9027	0.9685
with momentum 0.9	0.9349	0.0094	0.9173	0.9645
Adam	**0.9485**	0.0072	0.9360	0.9792



Training loss for the three optimisers from an identical start. Adam's advantage is not that it reaches a lower loss but that it reaches it from every start.

Read the spread before the mean. Adam's mean is about a point above plain descent; its **worst run is more than three points above**, and its spread is less than half. What Adam bought was the disappearance of the bad case — and insuring against the bad case is exactly what notebook 01 spent five restarts doing. Adam in one run matches what plain descent needed five to reach.

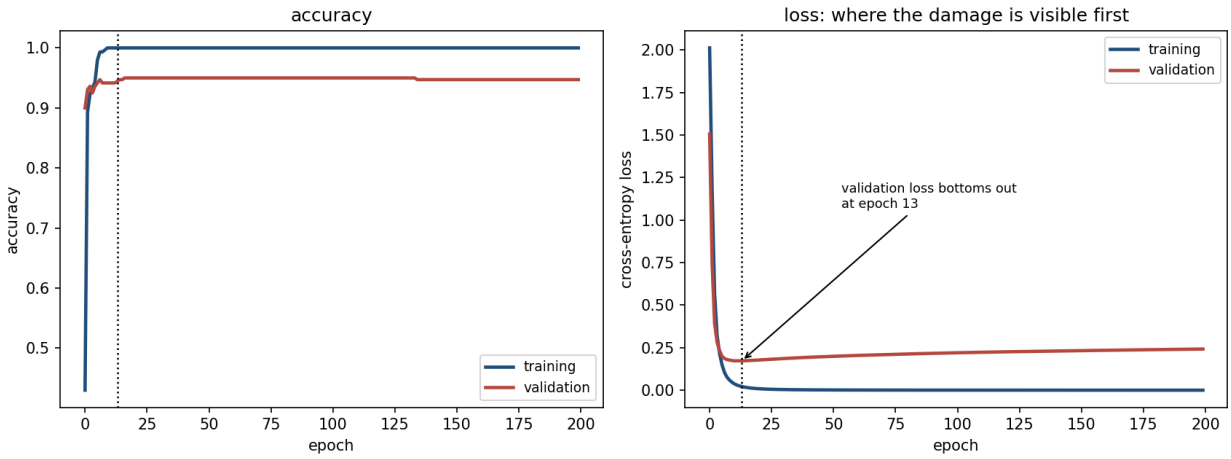
Momentum, on this problem, did nothing at all — a fraction of a point below plain descent, well inside either method's spread. That is worth reporting rather than quietly dropping. Momentum is a good default, not a guarantee, and a comparison in which every row improves on the last is usually a comparison that has been curated.

---

10. Regularisation, and knowing whether it worked

10.1 A network that memorises, and generalises anyway

Two hidden layers of 512 units on a deliberately small training set — 300 digits — gives **301,066 parameters for 300 examples**: 1,004 parameters per training example.



Left: accuracy. Right: cross-entropy loss, where the damage shows first. The training set is memorised perfectly, validation accuracy holds near 0.95, and validation loss bottoms out early and then climbs for the rest of training.

Two things happen and only one of them is a problem.

The network memorises the training set. Training accuracy reaches 1.0000 and training loss goes to essentially zero. With a thousand parameters per example there is more than enough freedom to store the answers outright.

And it generalises anyway, holding validation accuracy at 0.9472 with a best of 0.9500. Classical bias–variance reasoning from lesson 5 does not lead you to expect this, and it is nevertheless how large networks behave. Taking it seriously is an open research question; taking it as licence to stop validating would be a serious mistake.

What *does* degrade is the validation **loss**, which turns upward while validation accuracy stays flat. The network is not getting more answers wrong; it is getting steadily more confident about the ones it already has wrong. Accuracy cannot see that and cross-entropy can — which is the argument for early-stopping on the loss rather than on accuracy.

10.2 Three interventions

Early stopping monitors a validation metric and keeps the weights from the best epoch. It is the cheapest regulariser there is, it needs no hyperparameter beyond patience, and it is the one to reach for first.

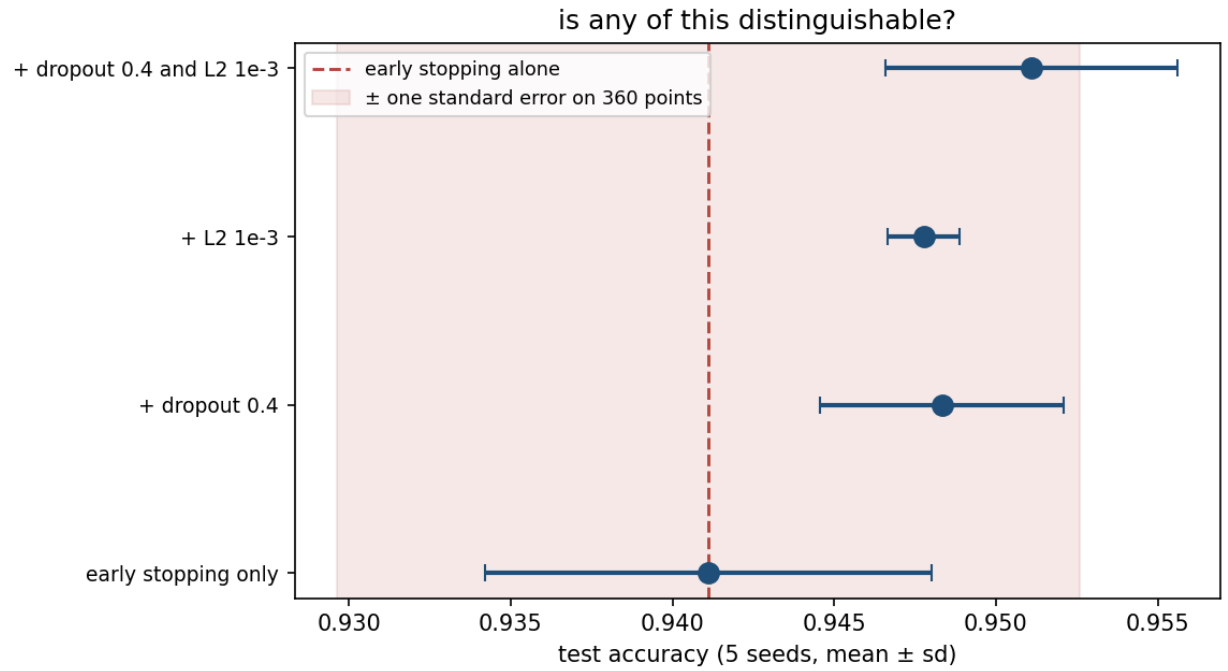
L2 regularisation adds $\frac{\lambda}{2}\|W\|^2$ to the cost, which adds λW to every weight gradient — each step shrinks the weight slightly toward zero before the data's gradient moves it. Identical in form to Ridge in lesson 3, and in this context usually called weight decay.

Dropout deletes each unit independently with probability p on every training batch, and rescales the survivors by $1/(1-p)$ so the expected input to the next layer is unchanged. At prediction time nothing is dropped. The usual explanation is that a unit cannot rely on any particular other unit being present, so the layer cannot build fragile co-adaptations; an equivalent reading is that it trains an ensemble of exponentially many thinned networks sharing weights, which connects it to lesson 7's bagging.

10.3 Whether any of it is distinguishable

Trained with early stopping on validation, best weights restored, scored on the **test** set that nothing was chosen against, five seeds each:

method	test accuracy	sd	train − test gap	epochs run
early stopping only	0.9411	0.0069	0.0589	38.8
+ dropout 0.4	0.9483	0.0038	0.0503	48.8
+ L2 10^{-3}	0.9478	0.0011	0.0522	198.4
+ dropout 0.4 and L2 10^{-3}	**0.9511**	0.0045	0.0489	161.6



The four configurations with their seed-to-seed spread, against a band of one standard error on 360 test examples. The differences are real and small, and this experiment cannot rank them.

All three land between 0.7 and 1.0 points above early stopping alone, against a seed-to-seed spread of 0.1 to 0.7 points. That is one to two standard deviations: **enough to say they helped, nowhere near enough to rank them.** On a single run with one seed you could comfortably have measured the three in any order.

Note the `epochs run` column, which says something the accuracy column cannot: L2 kept the validation loss improving for roughly four times as long before early stopping fired. A real qualitative difference, in a comparison whose headline numbers are not separable.

Reporting it this way is not excessive caution. Four numbers with no spread beside them would have supported a confident sentence about which regulariser is best, and that sentence would have been unfounded. This is lesson 5's discipline applied to a lesson-9 method, and it is the part of both lessons you are most likely to be asked to defend out loud.

10.4 The intervention that is not a hyperparameter

training examples	test accuracy	sd
100	0.9028	0.0060
200	0.9426	0.0047
300	0.9463	0.0035
500	0.9694	0.0039
800	0.9722	0.0000
1,077	**0.9787**	0.0026



Test accuracy against training-set size, architecture and protocol fixed, on a logarithmic axis. The dashed line is the best regularised result on 300 examples.

Going from 300 examples to 1,077 is worth **+3.2 points**. The best regulariser at 300 examples was worth **+1.0**. More data won by a factor of three, and unlike every method in section 10.2 it is not something you can tune your way to.

---

11. How many units? Capacity against optimisation

11.1 A yardstick made of lines

Section 4.1 established that column j of $W^{[1]}$ is a line. A hidden layer of H units therefore draws H lines, and the output unit takes a weighted vote over which side of each a point falls. So a natural question: how well could H lines possibly fence a circle?

For a regular H-sided polygon this can be computed exactly rather than searched for. In polar coordinates a regular polygon of apothem a (the distance from its centre to the middle of each side) has boundary $r(t) = a/\cos t$ for $|t| \le \pi/H$, repeated H times. For a standard two-dimensional normal, $P(r < s) = 1 - e^{-s^2/2}$ (section 2.2), so the polygon and the true circle of radius R disagree, at angle t, on an annulus of probability $\left|e^{-\min^2/2} - e^{-\max^2/2}\right|$ where min and max are the smaller and larger of R and $r(t)$. Integrating over one sector:

$$\text{accuracy}(a) = 1 - \frac{H}{2\pi} \int_{-\pi/H}^{\pi/H} \left| e^{-\min(R,\, r(t))^2/2} - e^{-\max(R,\, r(t))^2/2} \right| dt$$

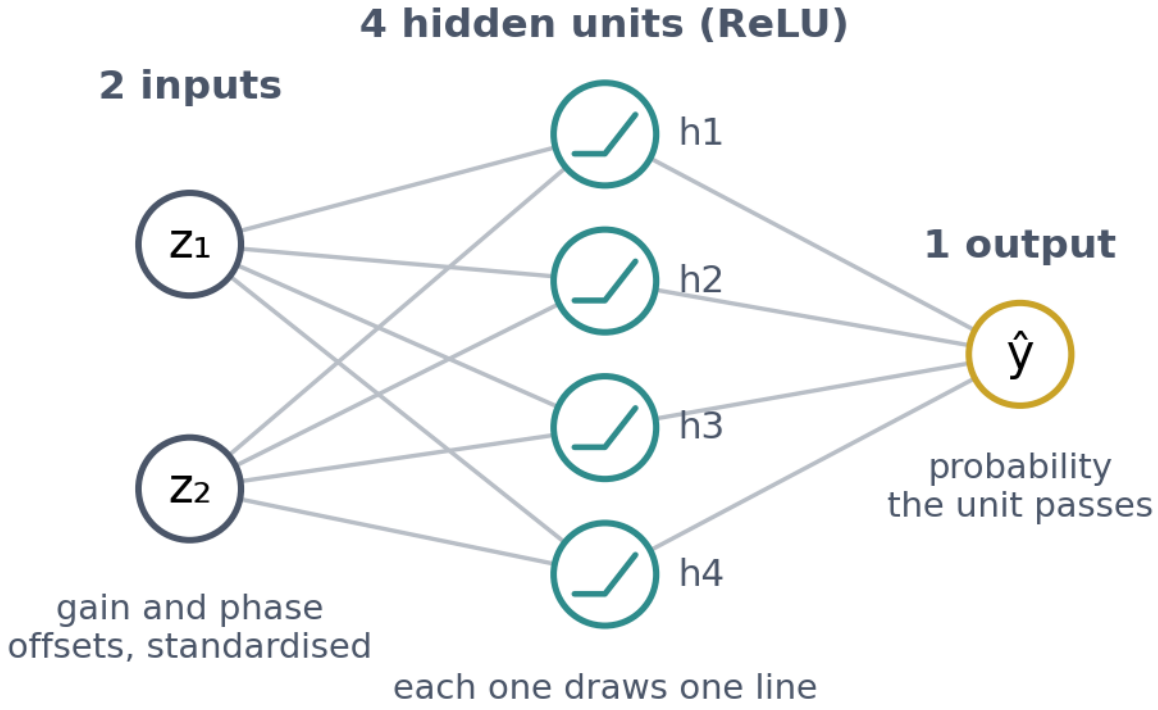
and maximising over a gives the best regular H-gon. Converting to accuracy against the *recorded* labels uses the rig-error identity of section 11.3.

Notebook 01 evaluates this integral and, as an independent check, scores the same polygons against 400,000 sampled points. The two agree to four decimal places: for the 4-gon, 0.9396 against 0.9392; for the 8-gon, 0.9859 against 0.9858; for the 16-gon, 0.9965 against 0.9965.

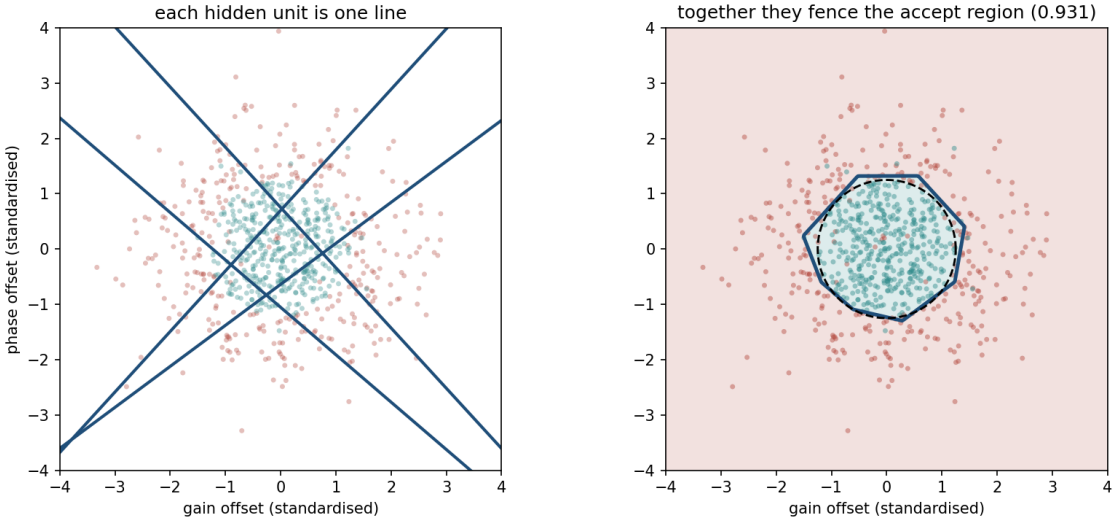
One caveat on the table in section 11.2, since it mixes two kinds of number. The entries from three lines upward are this integral, and are exact. The one- and two-line entries are a half-plane and a strip, which have no such tidy closed form and are estimated by sampling instead — so they carry an uncertainty of about ± 0.001, and their fourth decimal should not be read. `Docs/worked_examples.py` recomputes the half-plane by quadrature and gets 0.6500 against the table's 0.6491, which is exactly that uncertainty and not a disagreement.

11.2 What the sweep shows

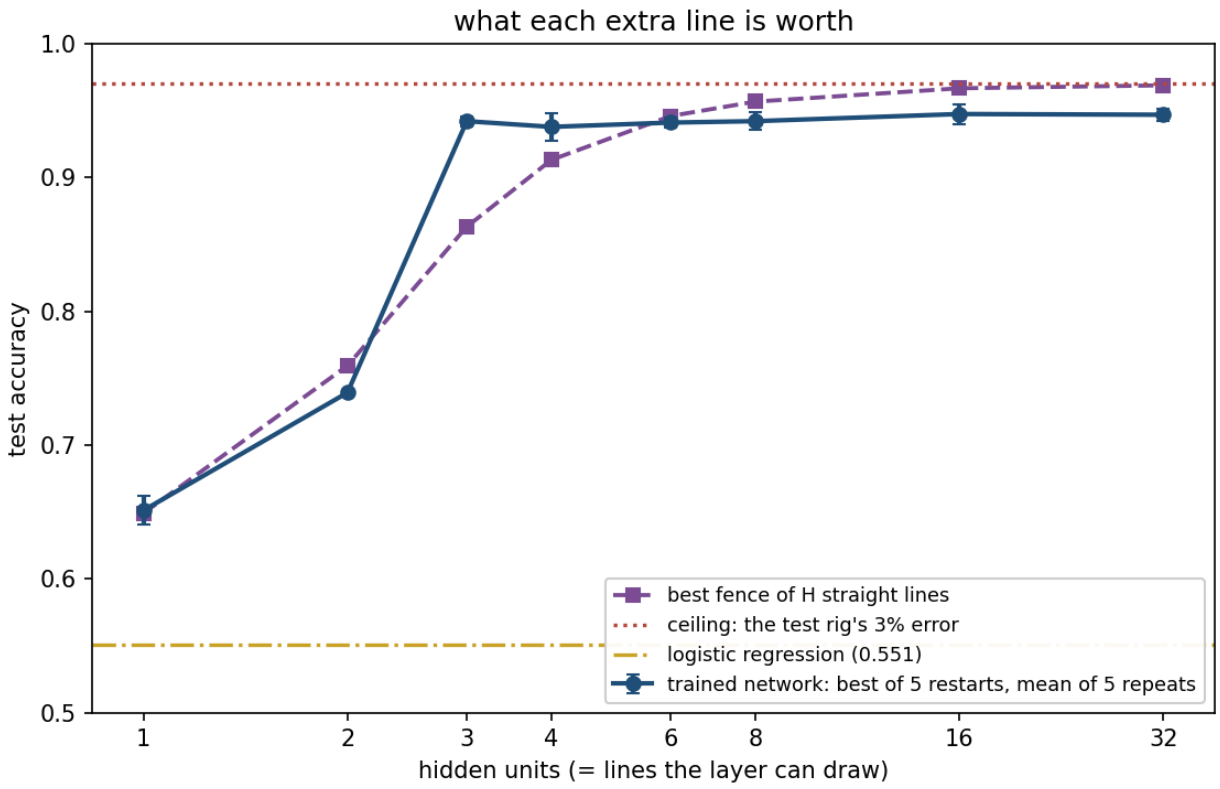
The question this section asks is how many hidden units the drift problem actually needs, and the answer is worth having in a form you can picture. Each hidden unit draws one straight line; the output layer combines them. So "how many units" is really "how many straight lines does it take to fence off a circle", and that is a question with a geometric answer as well as a measured one. The two figures below are those two answers, and the table underneath checks them against each other.



The network of this section: two inputs, the standardised gain and phase offsets; four hidden ReLU units, each of which draws one straight line in the plane of the inputs; one output, the probability that the unit passes. The weights are trained, not written by hand, so the connections carry no numbers.



Left: the four lines a trained four-unit hidden layer draws, plotted straight from the columns of $W^{[1]}$. Right: the decision region they produce together, with the true circle dashed. Four lines enclose a region; one line cannot.



Test accuracy against hidden-layer width, with the best fence of the same number of lines and the rig's ceiling. The steep part is over by three units.

hidden units	trained	sd	best fence of H lines	vs the true rule
1	0.6512	0.0106	0.6491	0.6621
2	0.7392	0.0016	0.7593	0.7592
3	0.9421	0.0037	0.8632	0.9685
4	0.9379	0.0101	0.9132	0.9664
6	0.9411	0.0039	0.9460	0.9696
8	0.9421	0.0068	0.9567	0.9717
16	**0.9475**	0.0074	0.9667	0.9760
32	0.9469	0.0047	0.9689	0.9776

Four readings, in order of how much they matter.

Almost everything arrives in the first three lines. One unit scores what the best single line scores. The step from two units to three is worth 20 points; everything from three units to thirty-two is worth 0.5.

The polygon is a floor, not a ceiling. At three and four units the trained network *beats* the best regular polygon of the same number of sides, by 7.9 points at three. Three lines cut the plane into seven regions, not one triangle, and a weighted vote over them can select shapes an intersection of half-planes cannot. So the fence curve says what H lines are *at least* worth.

Past six units the binding constraint changes. The fence curve keeps climbing toward 0.97 as more sides become available; the trained network does not follow, settling just below 0.95. The lines are there and gradient descent does not put them to work. That gap is the optimiser — the one section 9.3's Adam narrowed, by removing the bad runs rather than raising the good ones.

The score you can see is not the score you have. The last column is the same networks measured against `truly_within_tolerance` instead of the recorded verdict, and it runs about three points higher throughout.

11.3 The two ceilings, and the identity connecting them

If a classifier agrees with the true rule on a fraction q of units, and the rig independently records the wrong verdict with probability ε, the two agree exactly when both are right or both are wrong:

$$\text{accuracy against recorded labels} = q(1-\varepsilon) + (1-q)\varepsilon$$

At $q = 1$ this gives $1 - \varepsilon = 0.97$, the ceiling quoted throughout. Notebook 01 checks the identity across the whole sweep and finds a largest discrepancy of **0.0044** — the residual being that the model's errors are not quite independent of the rig's, since both concentrate near the boundary.

Substituting the best row: the 16-unit network agrees with the true acceptance rule on 97.60% of units, so its expected measured accuracy is $0.9760 \times 0.97 + 0.0240 \times 0.03 = 0.9474$, against 0.9475 observed.

This is the honest reading of the whole lesson's headline score. The network learned the true boundary to within 2.4%. What it is *scored* at is 0.9475, and the 2.9-point difference is a property of the measuring instrument. On any real dataset only the lower number exists, and there is no way from inside the data to tell how much of the shortfall is yours.

---

12. Choosing, and what carries into lesson 10

If	then
the boundary depends on inputs *in combination*	a hidden layer earns its cost
the inputs vote roughly independently	try the linear model first; digits lost only 3.5 points
the network sits at chance	check the learning rate before the architecture
training accuracy is high, validation low	early stopping first, then more data, then dropout or L2
both accuracies are low and still improving	it is underfitting: α too small, or too few epochs
a deep network will not train	check the activation, then the initialisation
you wrote the backward pass yourself	gradient-check it, before anything else

Lesson 10 changes exactly one assumption. Everything here treats the 64 pixels of a digit as 64 unrelated numbers — permute them consistently across the dataset and nothing in this lesson notices. That is obviously wrong for an image, where a pixel's neighbours are the most informative thing about it, and the architecture that encodes it is the convolutional network.

Summary

- **A neuron is a line.** A hidden layer of H units is H lines, and the output unit votes over them. Three lines enclose a region; one never does.
- On the acceptance data that is the difference between 0.55 and 0.94. On handwritten digits it is the difference between 0.93 and 0.97 — **a hidden layer is not always the answer.**
- A hidden layer does not classify. It **moves the data** until the last layer's single line suffices.
- **Backpropagation is the chain rule applied right to left**, reusing the forward pass, at about the cost of one extra forward pass. Sigmoid with cross-entropy, and softmax with cross-entropy, both collapse to $\hat{y} - y$ at the output.
- **Gradient-check anything you differentiated by hand.** Four lines, and the only defence against a wrong gradient that trains anyway.
- **A sigmoid layer divides the gradient by about four** — its derivative never exceeds ¼. Six layers of it compound to roughly 3,500, and a six-layer sigmoid network never leaves chance. This is the number to remember.
- Weights cannot start at zero: an all-zero network is frozen but for one bias, and converges to the label entropy, 0.6887. They cannot start large either — 73% of an eight-layer tanh stack saturates.
- Capacity is necessary, not sufficient: a two-unit solution to the drift problem exists and gradient descent finds it in 4 runs of 20.
- Report differences with the spread they were measured against. Dropout and L2 each bought under a point here, against a seed spread of up to 0.7. More data bought 3.2.

Homework

Exercise 9 — see `Exercises/09_neural_networks.md`. Discussed at the start of lesson 10, **Friday 4 December 2026**.

Notation used in this lesson

Symbol	Meaning
X	design matrix, $m \times n$, one example per row
m, n	number of examples, number of input features
H	number of hidden units
K	number of classes
$W^{[l]}, b^{[l]}$	weights and biases of layer l
$Z^{[l]}, A^{[l]}$	pre-activations and activations of layer l
g, σ	hidden activation function; the sigmoid specifically
$\hat{y}$, y	prediction, target
$\delta^{[l]}$	$\partial J / \partial Z^{[l]}$, the backpropagated signal
J	the cost being minimised
α	learning rate, and nothing else
λ	L2 regularisation strength
$\odot$	elementwise (Hadamard) product

Further reading

Source	Why
Goodfellow, Bengio & Courville, *Deep Learning* (2016), ch. 6	The standard derivation of backpropagation, in the column convention
Rumelhart, Hinton & Williams, "Learning representations by back-propagating errors", *Nature* 323 (1986)	The paper that made networks trainable; six pages, still readable
Glorot & Bengio, "Understanding the difficulty of training deep feedforward neural networks" (2010)	Where section 8.2 comes from, with the variance argument in full
He et al., "Delving deep into rectifiers" (2015)	The factor of 2 for ReLU, and why it matters at depth
Srivastava et al., "Dropout: a simple way to prevent neural networks from overfitting", *JMLR* 15 (2014)	Dropout's original motivation and its ensemble reading
Zhang et al., "Understanding deep learning requires rethinking generalization" (2017)	Why section 10.1's memorising-and-generalising network is still an open problem